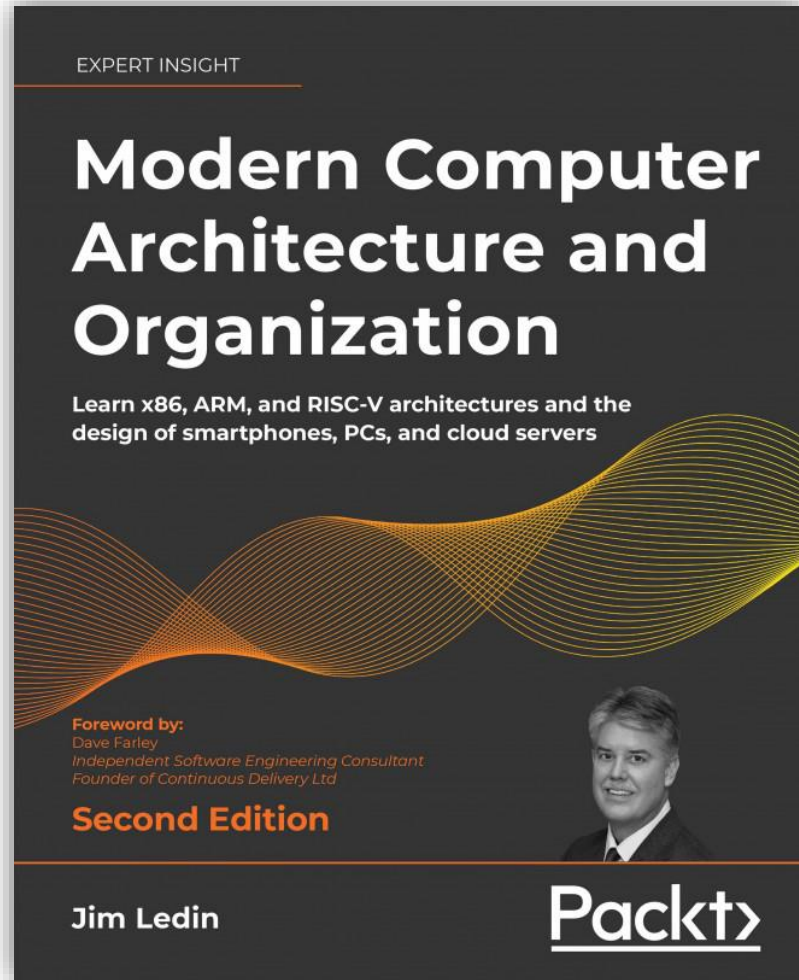


Computer Architecture

Chapter 5 – Hardware-Software Interface



VIVES University of Applied Sciences Bachelor in electronics-ICT



— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. **Device drivers**
2. BIOS
3. The boot process
4. Operating systems
5. Processes and threads
6. Multiprocessing

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers

- A **device driver** provides a standardized interface for software applications to interact with a category of peripheral devices.
- This avoids the need for the developer of each application to understand and implement all of the technical details required for the proper operation of each type of device.
- Most device drivers allow multiple simultaneously executing applications to interact with multiple instances of associated peripherals in a **secure** and **efficient** manner.
- At the lowest level of interaction, device driver code provides software instructions that manage communication interactions with the peripheral, including handling interrupts generated by device service requests.
- A device driver controls the operation of **hardware resources** provided by the **processor**, the **peripheral device**, and by **other system components** such as the **processor chipset**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers

- In computer systems supporting **privileged execution modes**, device drivers usually operate at an *elevated privilege level*, which grants access to peripheral interfaces that are inaccessible to less privileged code.
- This ensures that only trusted code is permitted to interact with these interfaces directly.
- If **unprivileged** application code were able to access a peripheral's hardware interface, a programming error that caused the device to behave improperly would immediately affect all applications that attempt to use the device.
- The steps involved in transitioning the flow of instruction execution and data between unprivileged user code and privileged driver code will be introduced in *Chapter 9, Specialized Processor Extensions*.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers

- As we learned in *Chapter 3, Processor Elements*, the two principal methods for accessing I/O devices are **port-mapped I/O** and **memory-mapped I/O**.
- Although **memory-mapped I/O** is predominant in modern computers, some architectures—such as x86—continue to support and use **port-mapped I/O**. In an x86 system, many modern peripheral devices provide an interface that **combines** port-mapped I/O and memory-mapped I/O.
- Programming tools for modern operating systems, such as those available for **Linux** and **Windows**, provide **resources for developing device drivers** capable of interacting with peripheral devices using port- and memory-mapped I/O techniques.
- **Installing a device driver** in these operating systems **requires elevated privilege**, but **users** of the driver do **not** require any special privileges.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers

- Although device drivers for sophisticated peripheral devices can be quite complex and difficult to understand for those not intimately familiar with the device's hardware and internal firmware, some legacy devices are fairly simple.
- One example is the **parallel printer port** introduced on early PCs, and a standard component of personal computers for many years.
- Even though modern computers rarely include these interfaces, **inexpensive parallel port expansion cards** remain readily available, and modern operating systems provide driver support for these interfaces.
- Electronics hobbyists frequently use a **parallel port** as a simple interface for interacting with external circuits using Transistor-Transistor Logic (TTL) 5V digital signals on PCs running **Windows** or **Linux**.

— Computer Architecture

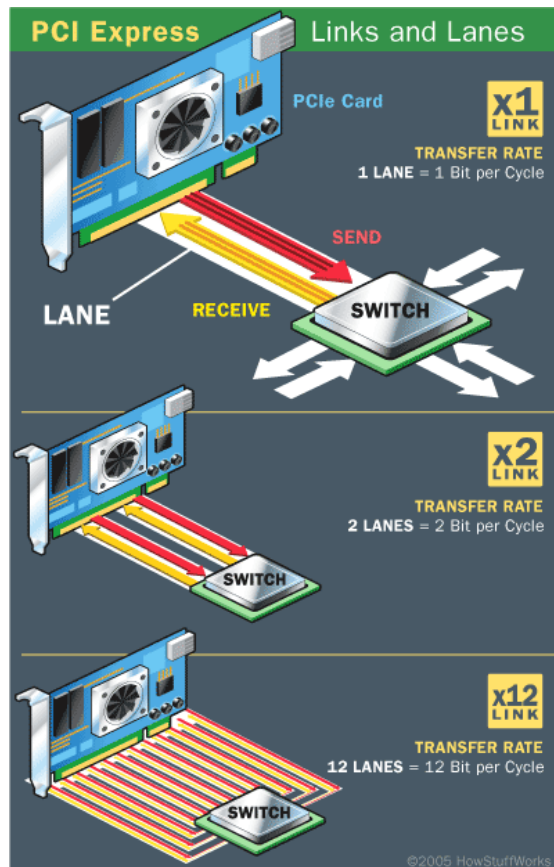
Chapter 5: Hardware-Software Interface

1. Device drivers: PCIe device drivers

- **PCIe** uses **high-speed serial connections** for communication between the **processor** and **PCIe peripheral devices**.
- PCIe was expressly designed to be software-compatible with the parallel **PCI bus** used in PCs of the 1990s. Device drivers written for PCI continue to work properly in computers using the serial PCIe bus.
- The task of translating between processor I/O instructions such as **in** and **out** and the sequential serial data transfers necessary to communicate with PCIe devices is handled transparently by the **PCIe bus controllers** in the processor, chipset, and PCIe devices.
- PCI and PCIe devices perform an **automated configuration operation** during **system startup** and when a device is hot plugged in a running system. **Hot plugging** is the installation of hardware in a system that is powered on.
- Once configuration is complete, the device interface is known to the operating system.
- The interface between a **PCI** or **PCIe** peripheral and the processor may include any combination of the following communication paths:
 - One or more **I/O port ranges**
 - One or more **memory regions** supporting **memory-mapped I/O**
 - Connection to a processor **interrupt handler**

Computer Architecture

Chapter 5: Hardware-Software Interface



©2005 HowStuffWorks

Figure 5.1: PCIe Links & Lanes

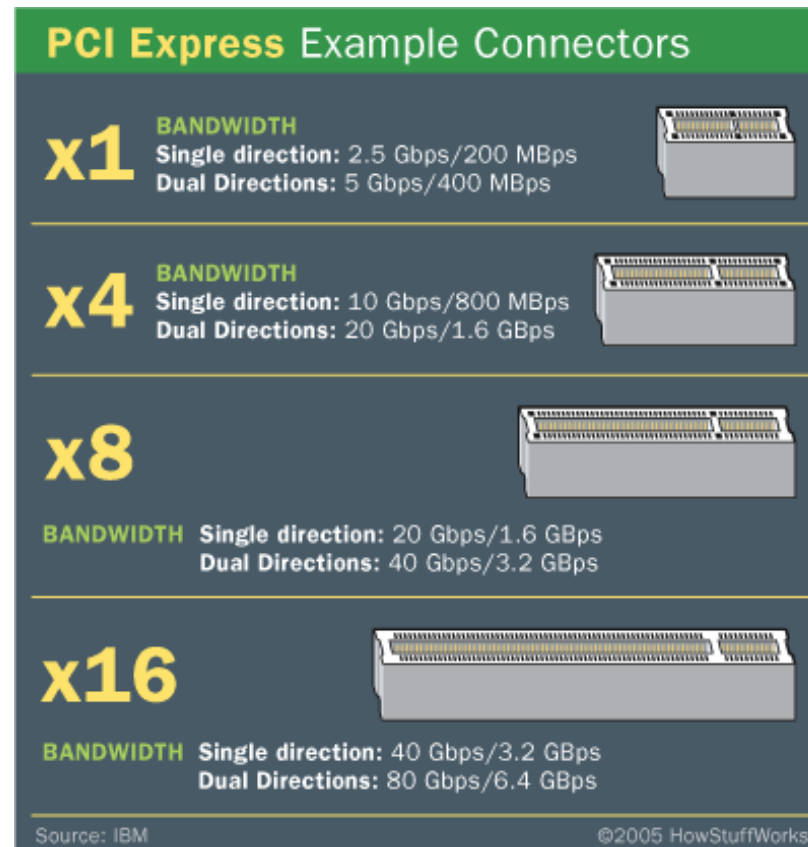


Figure 5.2: PCIe connectors

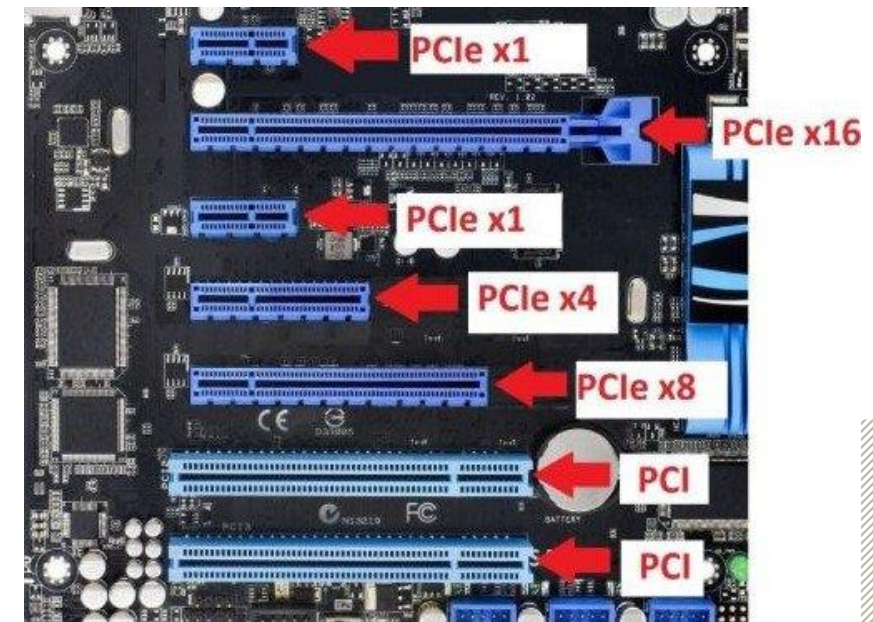


Figure 5.3: PCIe connectors

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: PCIe device drivers

- The interface configuration procedure applies to *both* **PCI** and **PCIe drivers**, enabling legacy PCI drivers to work properly in PCIe systems.
- Of course, the **physical** card interface differs greatly between parallel PCI and serial PCIe devices, so the cards themselves are not interchangeable across bus technologies.
- The **bus slots** for **PCIe** are intentionally different from PCI slots to prevent the accidental insertion of PCI devices into PCIe slots, and vice versa.
- Bulk data transfer to and from peripheral devices generally relies on **Direct Memory Access** (DMA) in both PCI and PCIe systems.
- In PCIe systems, DMA operations take full advantage of the high data rates possible with **multi-lane serial connections**, blasting data across the interface at close to the theoretical maximum speed each single- or multilane link can support.
- The technological evolution that supplanted legacy parallel bus PCI technology with the vastly higher-performing **multi-lane serial technology** of PCIe, while retaining seamless device driver compatibility, has been quite remarkable.

Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: PCIe device drivers

PCIe Topology Example

- PCIe: A tree structure with nodes connected to each other via point-to-point links.
- The root node is called the root complex (RC).
- The leaf nodes are called end points (EP) and the nodes that connect multiple devices to each other are called switches (SW).

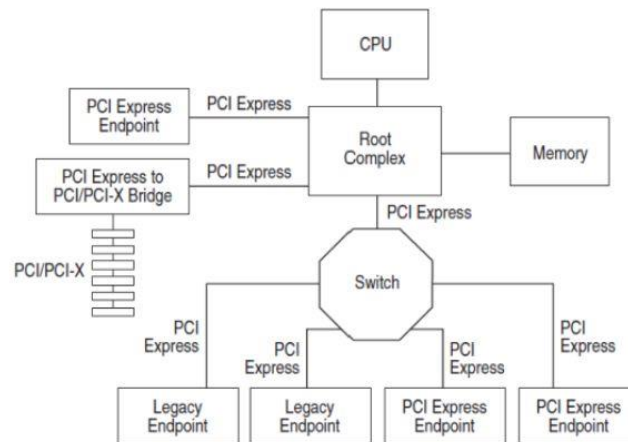


Figure 5.4: PCI express topology

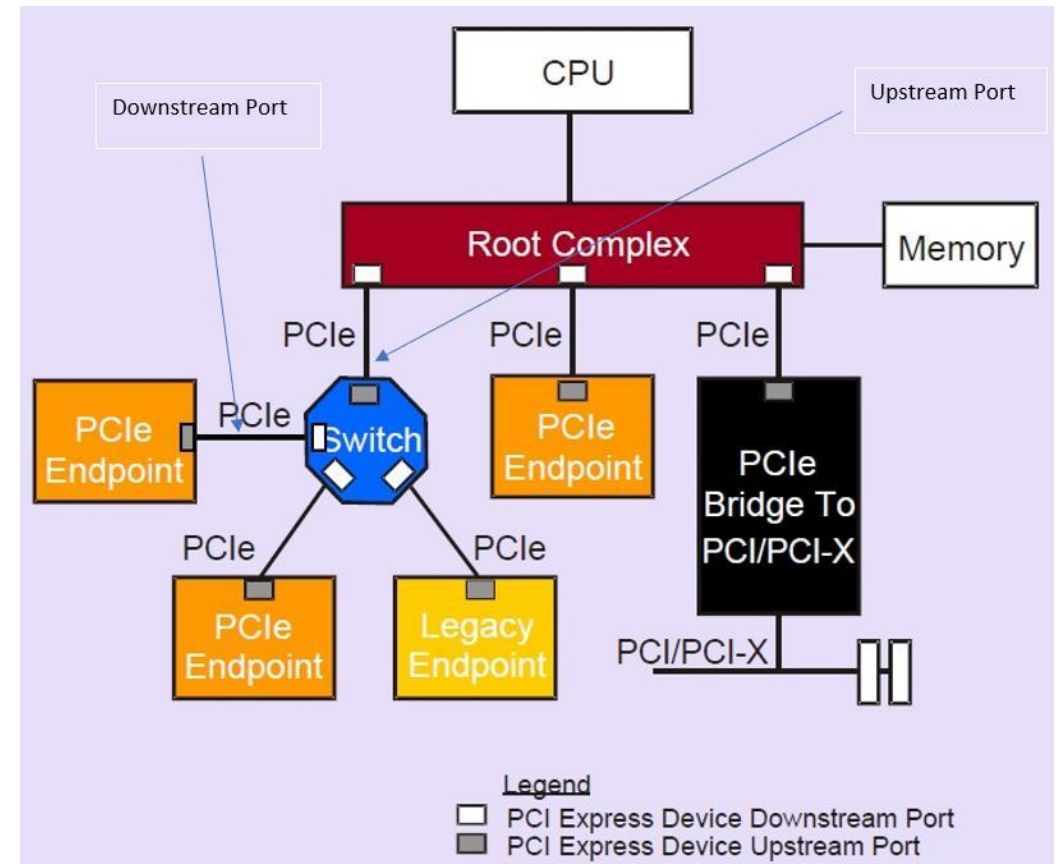


Figure 5.5: PCIe Topology Example

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

- A **device driver** is a **software module** that implements a set of *predefined functions*, enabling the operating system to associate the driver with compatible peripheral devices and perform controlled access to those devices.
- This allows system processes and user applications to perform I/O operations on **shared devices**.
- This section provides a brief overview of some of the most commonly used functions a **Linux device driver** must implement for use by application developers.
- This example will prefix the function names with the fictitious device name *mydevice* and is written in the C programming language.
- The following functions perform tasks related to **initialization** and **termination** of the driver itself:

```
int mydevice_init(void);  
void mydevice_exit(void);
```

Figure 5.6: device driver functions

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

- The following functions perform tasks related to **initialization** and **termination** of the **driver** itself:

```
int mydevice_init(void);  
void mydevice_exit(void);
```

Figure 5.6: device driver functions

- The operating system calls **mydevice_init** to **initialize** the device at **system startup** or at a later time if the device is connected by hot plugging. The **mydevice_init** function returns an integer code, indicating if the initialization was successful or, if unsuccessful, the error that occurred. **Successful driver initialization** is indicated by a return code of **zero**.
- When the driver is no longer needed, such as during system shutdown or when the device is removed while the system is running, **mydevice_exit** is called to **release** any **system resources** allocated by the driver.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

- The next pair of functions, shown here, allows system processes and user applications to **initiate** and **terminate communication sessions** with the device:

```
int mydevice_open(struct inode *inode, struct file *filp);  
int mydevice_release(struct inode *inode, struct file *filp);
```

Figure 5.7: device driver communication sessions functions

- mydevice_open** attempts to **initiate access** to the device and **reports** any **errors** that may occur in the process. The **inode** (= *index node*) parameter is a **pointer** to a data structure containing information required to access a specific file or other device.
- The **filp** parameter is a **pointer** to a data structure containing information about the open file.
- In **Linux**, all types of **devices** are consistently represented as **files**, even if the device itself is not inherently file-based. The name **filp** is short for **file pointer**.
- All functions operating on the file receive a pointer to this structure as an input.
- Among other details, the **filp** structure indicates whether the file is opened for **reading, writing, or both**.
- The **mydevice_release** function closes the device or file and **deallocates any resources** allocated in the call to **mydevice_open**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

- Following a successful call to **mydevice_open**, application code can begin to **read** from and **write** to the **device**. The functions performing these operations are as follows:

```
ssize_t mydevice_read(struct file *filp, char *buf,  
                      size_t count, loff_t *f_pos);  
ssize_t mydevice_write(struct file *filp, const char *buf,  
                       size_t count, loff_t *f_pos);
```

Figure 5.8: device driver read and write functions

- The **mydevice_read** function **reads** from the device or file and transfers the resulting data to a **buffer** in application memory space. The **count** parameter indicates the **requested amount of data**, and **f_pos** indicates the **offset from the start of the file** at which to begin reading.
- The **buf** parameter is the **address of the destination** for the data.
- The **number of bytes** actually **read** (which may be less than the number requested) is provided as the function return value, with a data type of **ssize_t**.
- The **mydevice_write** function has most of the same parameters as **mydevice_read**, except that the **buf** parameter is declared **const** (constant) because **mydevice_write** reads from the memory address indicated by **buf** and **writes** that data to the file or device.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

```
ssize_t mydevice_read(struct file *filp, char *buf,  
                      size_t count, loff_t *f_pos);  
ssize_t mydevice_write(struct file *filp, const char *buf,  
                       size_t count, loff_t *f_pos);
```

Figure 5.8: device driver read and write functions

- One point of interest in the implementation of these functions is that the **privileged** driver code **cannot** (or at least should not, if the system permits it) **access user memory directly**.
- This is to prevent driver code from accidentally or intentionally reading from or writing to inappropriate memory locations, such as **kernel space**.
- To avoid this potential problem, special functions named **copy_to_user** and **copy_from_user** are provided by the **operating system** for use by drivers to access **user memory**. These functions take the necessary steps to validate the user-space addresses provided in the function call before copying data.
- During **system power-up**, before the operating system can boot and initialize its drivers, **firmware** must execute to perform low-level self-testing and system configuration. The next section presents an introduction to the code that first executes when the computer receives power: the **BIOS**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

1. Device drivers: Device driver structure

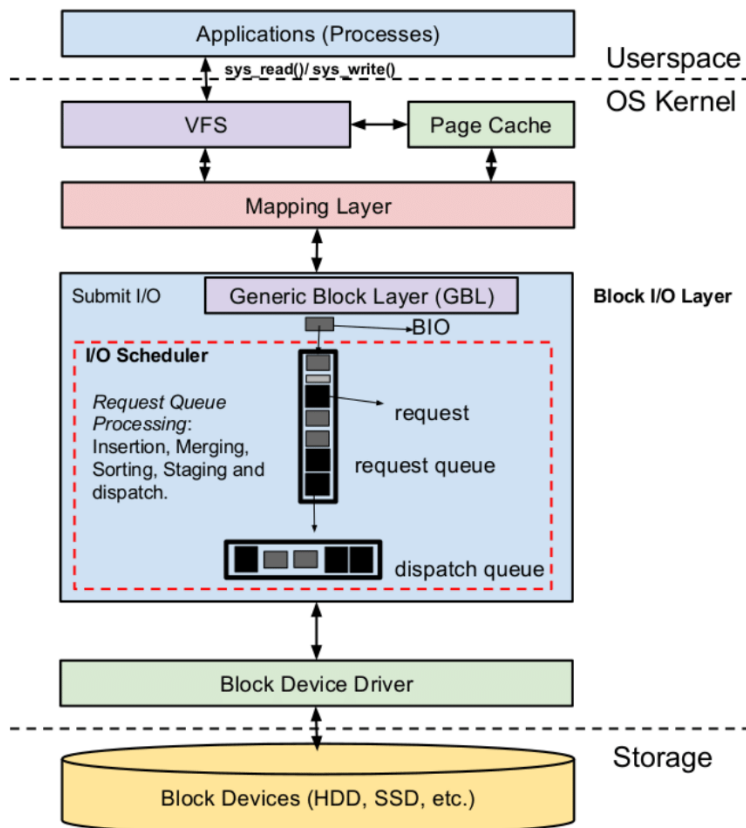


Figure 5.9: Architecture of Linux Kernel I/O stack

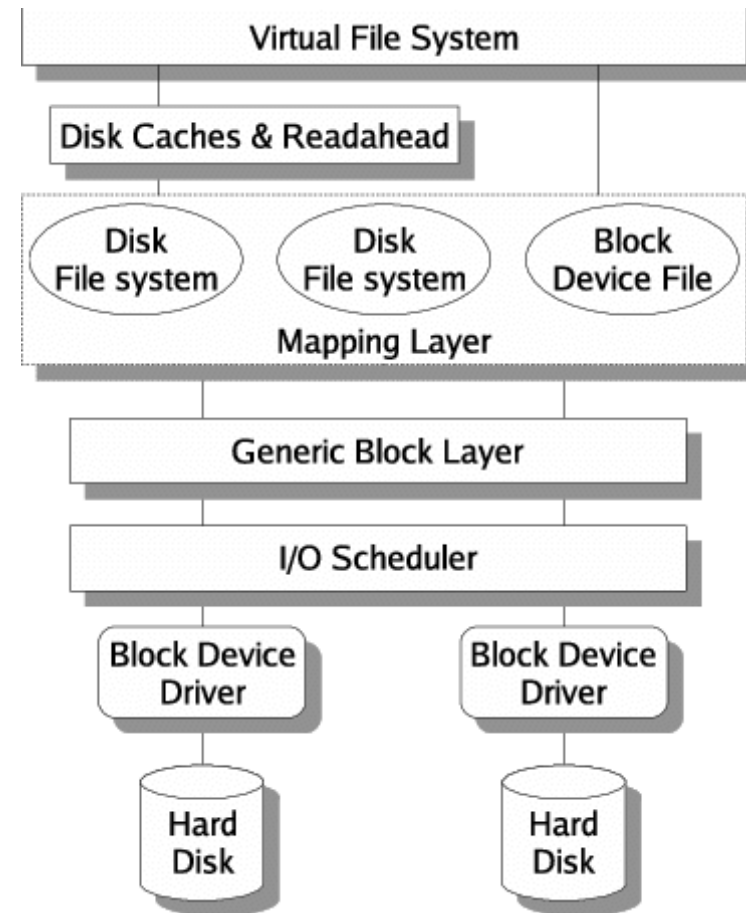


Figure 5.10: Linux I/O Subsystem

— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. Device drivers
- 2. BIOS**
3. The boot process
4. Operating systems
5. Processes and threads
6. Multiprocessing

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS

- A computer's **BIOS** (Basic Input Output System) contains the code that first executes at **system startup**.
- In the early days of personal computers, the BIOS provided a set of **programming interfaces** that abstracted the details of peripheral interfaces such as *keyboards* and *video displays*.
- In modern PCs, the BIOS performs **system testing** and **peripheral device configuration** during **startup**. After that process has completed, the processor interacts with peripheral devices directly without further use of the BIOS.
- Early PCs stored the BIOS code in a read-only memory (**ROM**) chip on the motherboard. This code was permanently programmed and could not be altered.
- Modern motherboards generally store the motherboard BIOS in a **reprogrammable flash memory device**. This allows BIOS updates to be installed to add new features or to fix problems found in earlier firmware versions.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS

- One downside of BIOS reprogrammability is that this capability makes it possible for **malicious code** to be introduced into a system by writing to the BIOS flash memory. When this type of **attack** is successful, it enables the malicious code to execute every time the computer starts up. Fortunately, successful BIOS firmware attacks have proven to be quite rare.
- As the BIOS takes control during system startup, one of the first things it does is run a **Power-On Self-Test (POST)** of key system components. During this testing, the BIOS attempts to interact with system components including the *keyboard*, *video display*, and *boot device*, typically a disk drive.
- Although the computer may contain a high-performance graphics processor, the video interface used by the BIOS during startup is normally a primitive video mode, supporting *text display only*.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS

- The BIOS uses the **video** and **keyboard** interfaces to display any errors detected during system testing and allows the user to enter configuration mode and change stored settings.
- The keyboard and video interfaces provided by the BIOS enable the initial setup and configuration of a computer that does not yet contain a boot device.
- When the **video display** is **not working properly**, the BIOS will be unable to present information related to the error. In this situation, the BIOS attempts to use the **PC speaker** to indicate the **error**, using **a pattern of beeps**. Motherboard documentation provides information about the type of error indicated by each beep pattern.
- Depending on the system configuration, either the BIOS *or* the operating system manages the **initialization of PCI devices** during system startup. At the completion of a successful configuration process, all PCI devices have been assigned compatible I/O port ranges, memory-mapped I/O ranges, and interrupt numbers.
- As startup proceeds, the operating system identifies the **appropriate driver** to associate with **each peripheral** based on **manufacturer** and **device identification information** provided through **PCI/PCIe**.
- Following successful association, the driver interacts directly with each peripheral to perform I/O operations upon request.
- System processes and user applications call a **set of standardized driver functions** to initiate access to the device, perform read and write operations, and close the device.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS

- One common BIOS-related procedure is to select the **boot order** among the available storage devices.
- For example, this feature lets you configure the system to first attempt to boot from an *optical disk* containing a valid operating system image, if such a disk is in the drive.
- If *no bootable optical disk* is found, the system might then attempt to boot from the **main disk drive**. Several mass storage devices can be configured in priority order in this manner.
- BIOS configuration mode is accessed by pressing a specific key, such as **Esc** or the **F2** function key, during the early stage of the boot process. The appropriate key to press is usually indicated on screen during boot-up.
- Upon entering **BIOS configuration mode**, settings are displayed on screen in a menu format. You can select among different screens to modify parameters associated with features such as boot priority order.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS

- After making parameter changes, an option is provided to save the changes to **nonvolatile memory** (NVM) and resume the boot process. Be careful when doing this, because making inappropriate changes to the BIOS settings can leave the computer **unbootable**.
- The capabilities of BIOS implementations have grown substantially over the years since the introduction of the IBM PC. As PC architectures grew to support 32-bit and then 64-bit operating systems, the legacy BIOS architecture, however, failed to keep pace with the needs of the newer, more capable systems.
- Major industry participants undertook an initiative to define a system firmware architecture that left behind the limitations of the BIOS.
- The result of this effort was the **UEFI standard** (Unified Extensible Firmware Interface), which replaced the traditional BIOS capabilities in modern motherboards.

Computer Architecture

Chapter 5: Hardware-Software Interface

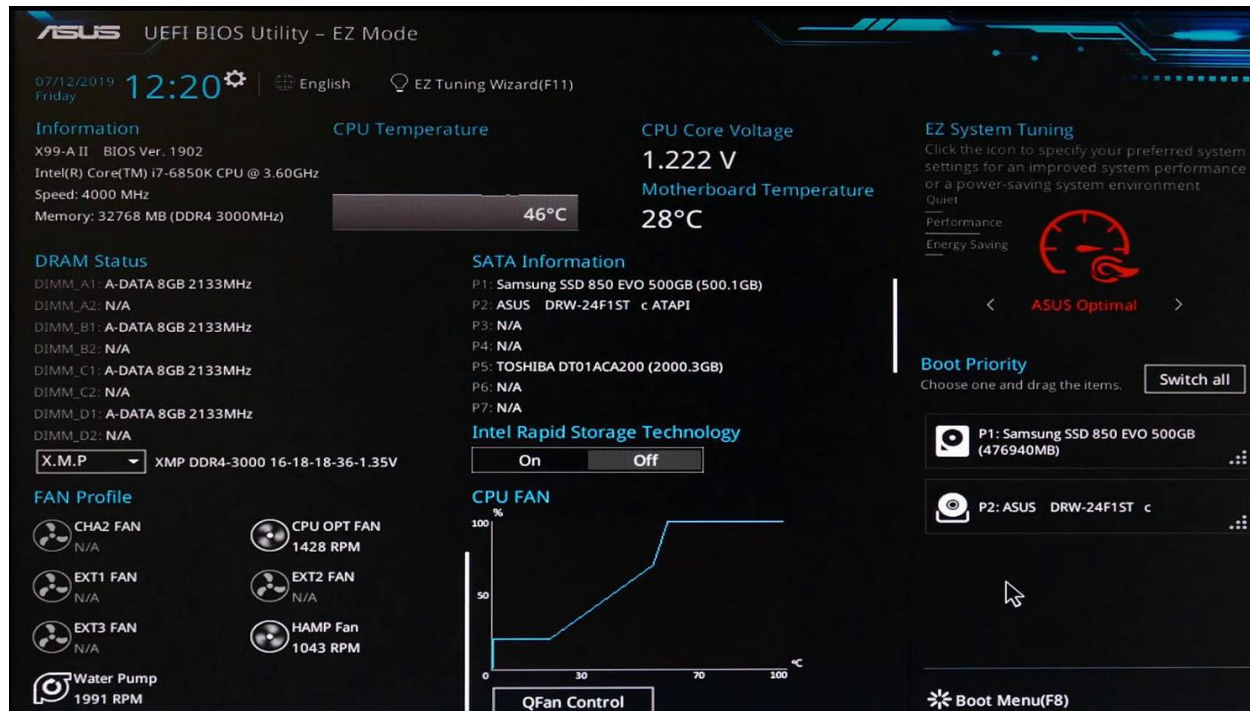


Figure 5.11: UEFI BIOS Utility

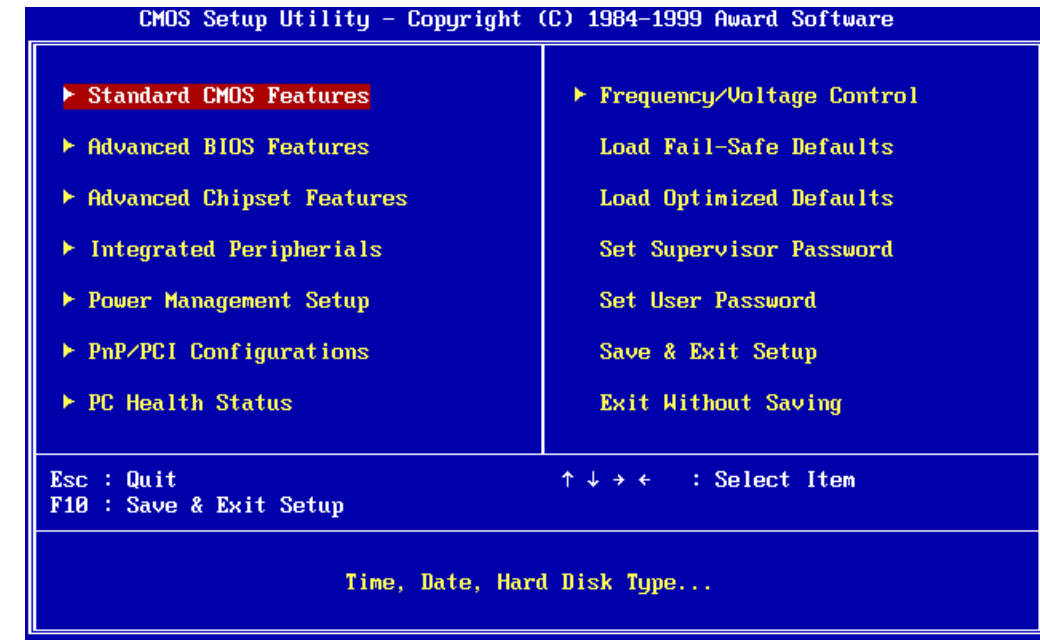


Figure 5.12: Legacy BIOS CMOS Setup Utility

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS: UEFI (Unified Extensible Firmware Interface)

- UEFI is a **2007 standard** defining an architecture for firmware that implements the functions provided by the legacy BIOS and adds several significant enhancements.
- As with BIOS, UEFI contains code executed immediately upon **system startup**.
- UEFI supports a number of design goals, including enabling **support for larger boot disk devices** (specifically, disk sizes greater than 2 terabytes (TB)), **faster start up times**, and **improved security** during the boot process.
- UEFI provides several features that, when enabled and used properly, substantially reduce the possibility of accidental or malicious corruption of firmware stored in UEFI flash memory.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS: UEFI (Unified Extensible Firmware Interface)

- In addition to the capabilities provided by legacy BIOS implementations described previously, the UEFI supports these features:
 - **UEFI applications** are executable code modules stored in **UEFI flash memory**.
 - UEFI applications provide extensions of capabilities available in the motherboard pre-boot environment and, in some cases, provide services for use by operating systems during runtime.
 - One example of a UEFI application is the **UEFI shell**, which presents a **command-line interface (CLI)** for interacting with the processor and peripheral devices.
 - The **UEFI shell** supports device data queries and permits modification of nonvolatile configuration parameters.
 - The **GNU GRand Unified Bootloader (GRUB)** is another example of a UEFI application. GRUB supports *multi-boot configurations* by presenting a menu from which the user selects one of multiple available operating system images to boot during system startup.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS: UEFI

- In addition to the capabilities provided by legacy BIOS implementations described previously, the UEFI supports these features:
 - **Architecture-independent device drivers** provide processor-independent implementations of device drivers for use by UEFI. This enables a single implementation of UEFI firmware to be used on architectures as diverse as **x86** and **Advanced RISC Machine (ARM)** processors.
 - Architecture-independent UEFI drivers are stored in a **byte-code format** that is interpreted by processor-specific firmware. These drivers enable UEFI interaction with peripherals such as *graphics cards* and *network interfaces* during the boot process.
 - **Secure Boot** employs **cryptographic certificates** to ensure that only legitimate device drivers and operating system loaders are executed during system startup.
 - This feature validates the digital signature of each firmware component before allowing it to execute. This validation process protects against many classes of malicious firmware-based attacks.

— Computer Architecture

Chapter 5: Hardware-Software Interface

2. BIOS: UEFI

- In addition to the capabilities provided by legacy BIOS implementations described previously, the UEFI supports these features:
 - **Faster booting** is achieved by performing operations in **parallel** that took place sequentially under the BIOS. In fact, booting is so much faster that many UEFI implementations do not offer the user an option to press a key during boot because waiting for a response would delay system startup.
 - Instead, operating systems such as Windows enable entry to UEFI settings by allowing the user to request access while the operating system is running, followed by a reboot to enter the UEFI configuration screen.
- The UEFI does not simply replace the functions of the old BIOS. It is a **miniature operating system** that supports advanced capabilities, such as allowing a remotely located technician to use a network connection to troubleshoot a PC that refuses to boot.

— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. Device drivers
2. BIOS
- 3. The boot process**
4. Operating systems
5. Processes and threads
6. Multiprocessing

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process

- The procedure for booting a *system image* varies, depending on the partition style of the mass storage device containing the **image**.
- Beginning in the early 1980s, the standard disk partition format was called the **master boot record (MBR)**.
- An **MBR partition** has a **boot sector** located at the logical beginning of its storage space. The MBR boot sector contains information describing the device's **logical partitions**. Each partition contains a filesystem organized as a *tree structure of directories and the files within them*.
- Due to the fixed format of MBR data structures, an MBR storage device can contain a **maximum of four logical partitions** and can be **no larger than 2 TB in size**, equal to 2^{32} 512-byte data sectors. These limits have become increasingly constraining as commercially available disk sizes grew beyond 2 TB.
- To resolve these issues, and in tandem with the development of **UEFI**, a **new partition format** called **GUID partition table (GPT)** (where **GUID** stands for **globally unique identifier**) was developed to eliminate restrictions on disk size and the number of partitions, while providing some additional enhancements.
- A GPT-formatted disk has a maximum size of 2^{64} 512-byte sectors, accommodating over **8 billion TB** of data. As normally configured, GPT supports up to **128 partitions per drive**.
- The type of each partition is indicated by a **128-bit GUID**, allowing an effectively unlimited number of new partition types to be defined in the future. Most users do not need very many partitions on a single disk, so the most obvious user benefit of GPT is its support for larger drives.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: BIOS boot

- Following **POST** and **device configuration**, BIOS begins the **boot process**.
- BIOS attempts to boot from the first device in the configured priority sequence.
- If a valid device is present, the firmware reads a small piece of executable code called the **boot loader** from the **MBR boot sector** and transfers control to it.
- At that point, the BIOS firmware has completed execution and is no longer active for the duration of system operation.
- The **boot loader** initiates the process of **loading** and starting the **operating system**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: BIOS boot

- If a **boot manager** is used with a **BIOS** motherboard, the **MBR boot sector code** must start the manager rather than loading an operating system directly.
- A **boot manager** (such as **GRUB**) presents a list from which the user selects the desired **operating system image**.
- The BIOS firmware itself has **no knowledge** of **multi-booting**, and the boot manager operating system selection process takes place **without BIOS involvement**.

Multi-booting versus boot priority order

Multi-booting allows the user to select the desired operating system from a menu of available choices. This differs from the boot priority list maintained by the BIOS, which empowers the BIOS itself to select the first available operating system image.

Figure 5.13: Multi-booting versus boot priority order

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: UEFI boot

- After the **POST** and **device configuration** stages have completed (in a manner very similar to the corresponding BIOS steps), **UEFI** begins the **boot process**.
- In a **UEFI motherboard**, a **boot manager** may be displayed as part of the UEFI start up procedure. A **UEFI boot manager**, which is part of the **UEFI firmware**, presents a menu from which the user can select the desired operating system image.
- If the user does not select an operating system from the boot manager within a few seconds (or if no boot manager menu is displayed), the UEFI attempts to boot from the **first device** in the configured priority sequence.
- The UEFI firmware reads the **boot manager executable code** (which is *separate* from the **UEFI boot manager**) and **boot loader files** from configured locations on the system disk and executes these files during the start up process.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: UEFI boot

- The following screenshot shows portions of the system **boot configuration data** (BCD) information stored on a **Windows 11 system**.
- To display this information on your computer, you must run the **bcdedit** command from a command prompt with **Administrator privilege**:

C:\>bcdedit	
Windows Boot Manager	

identifier	{bootmgr}
device	partition=\Device\HarddiskVolume1
path	\EFI\Microsoft\Boot\BootMGFW.EFI
...	
Windows Boot Loader	

identifier	{current}
device	partition=C:
path	\Windows\system32\winload.efi
...	

Figure 5.14: result of bcdedit command (Windows 10)

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: UEFI boot

- In this example, the **Windows Boot Manager** is located at `\EFI\MICROSOFT\BOOT\BOOTMGFW.EFI`.
- This file is normally stored on a **hidden disk partition** and is not readily available for display in directory listings.
- The **Windows boot loader** is identified as `\WINDOWS\system32\winload.efi` and is located at `c:\Windows\System32\winload.efi`.
- Unlike personal computers, most embedded devices use a much simpler boot process that does not involve **BIOS** or **UEFI**.

```
C:\>bcdedit

Windows Boot Manager
-----
identifier          {bootmgr}
device              partition=\Device\HarddiskVolume1
path                \EFI\MICROSOFT\BOOT\BOOTMGFW.EFI
...

Windows Boot Loader
-----
identifier          {current}
device              partition=C:
path                \WINDOWS\system32\winload.efi
...
```

Figure 5.14: result of bcdedit command (Windows 10)

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: Trusted boot

- The possibility of malicious modification of the UEFI firmware or the on-disk software components, executed during system boot necessitates additional protection, provided by the **trusted boot process**.
- The goal of the **trusted boot process** is to perform the entire boot sequence while ensuring that all of the software executed is properly authorized and has not been modified.
- The boot software verifies the **authenticity** of **firmware** and **software** files by computing the **cryptographic hash** of the bytes comprising each software component and ensuring the resulting hash value is valid using a digital signature.
- A **cryptographic hash** function generates a “*fingerprint*” for a block of data of arbitrary length.
- The output value produced by a given cryptographic hash function is always the same length.
- For example, the **256-bit Secure Hashing Algorithm** (SHA-256) always produces a 256-bit output regardless of the length of its input data (URL: [SHA-2](#)).
- The feature that makes a cryptographic hash function **secure** is that any attempt to modify the data used as input to the function will cause the hash output to also be different.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: Trusted boot

- It is effectively impossible to identify a set of modifications to an input data block that will produce the same hash output as the original data block.
- *While it is theoretically possible to come up with a modified data set with the same SHA-256 hash as the original block, the computations required to do so would take longer than the Earth's remaining lifetime on the fastest supercomputer available today.*
- To create a **digital signature**, a software publisher must compute the hash value for the **firmware** or **software component memory image**.
 - The next step is to encrypt the hash using a **private signing key**.
 - This **private encryption key** is associated with a **publicly available decryption key**.
- The encrypted hash value forms the **digital signature** that is stored along with the firmware or software file.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: Trusted boot

- The **public key** is made available to users of the firmware or software.
- This key is used to decrypt the encrypted hash value, which can then be compared to the hash value computed from the file's data bytes during boot.
- The two hash values must match exactly to confirm the file is unmodified since it was digitally signed. As with the cryptographic hash algorithm, it is effectively impossible to alter the digital signature in a manner that would enable undetected modifications to the protected data file.
- While the public key is not intended to be kept secret, the users of the key must ensure that the key itself is from the trusted source and has not been replaced by a malicious actor.
- To ensure that **only trusted public keys** are used during the boot process, these keys are *stored in a hardware location* that is inaccessible to unauthorized users and software.
- The standard mechanism for this in PCs is called the **Trusted Platform Module** (TPM).
- A TPM is, in effect, a microcontroller carefully designed to protect cryptographic keys.
- In newer computers, TPM keys are configured by the system manufacturer to ensure the system boots in a secure manner from the moment of its initial power-on.

— Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process: Embedded devices

- Most embedded systems, such as **smartphones**, do **not** generally have separate boot firmware such as the **BIOS** or **UEFI** in a PC.
- As we saw with the 6502, these devices perform a **processor hardware reset** when power is turned on and begin code execution at a **specified address**. All code in these devices is typically located in a nonvolatile storage region such as flash memory.
- During startup, **embedded devices** perform a sequence of events similar to the PC boot process. Peripheral devices are tested for proper operation and initialized prior to first use.
- The **boot loader** in such devices may need to select among multiple memory partitions to identify an appropriate system image.
- As with **UEFI**, embedded devices often incorporate **security features** during the boot process to ensure that the boot loader and operating system image are authentic before allowing the boot process to proceed.
- In both PC and embedded systems, startup of the boot loader is the first step in bringing up the operating system.

Computer Architecture

Chapter 5: Hardware-Software Interface

3. The boot process

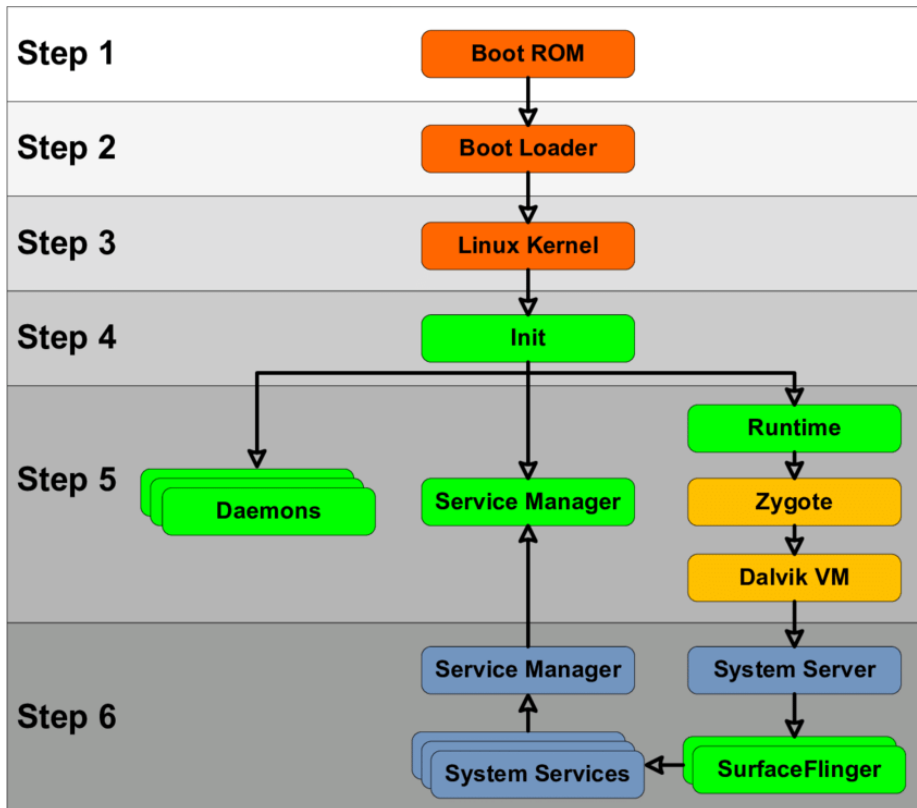


Figure 5.15: Android boot sequence

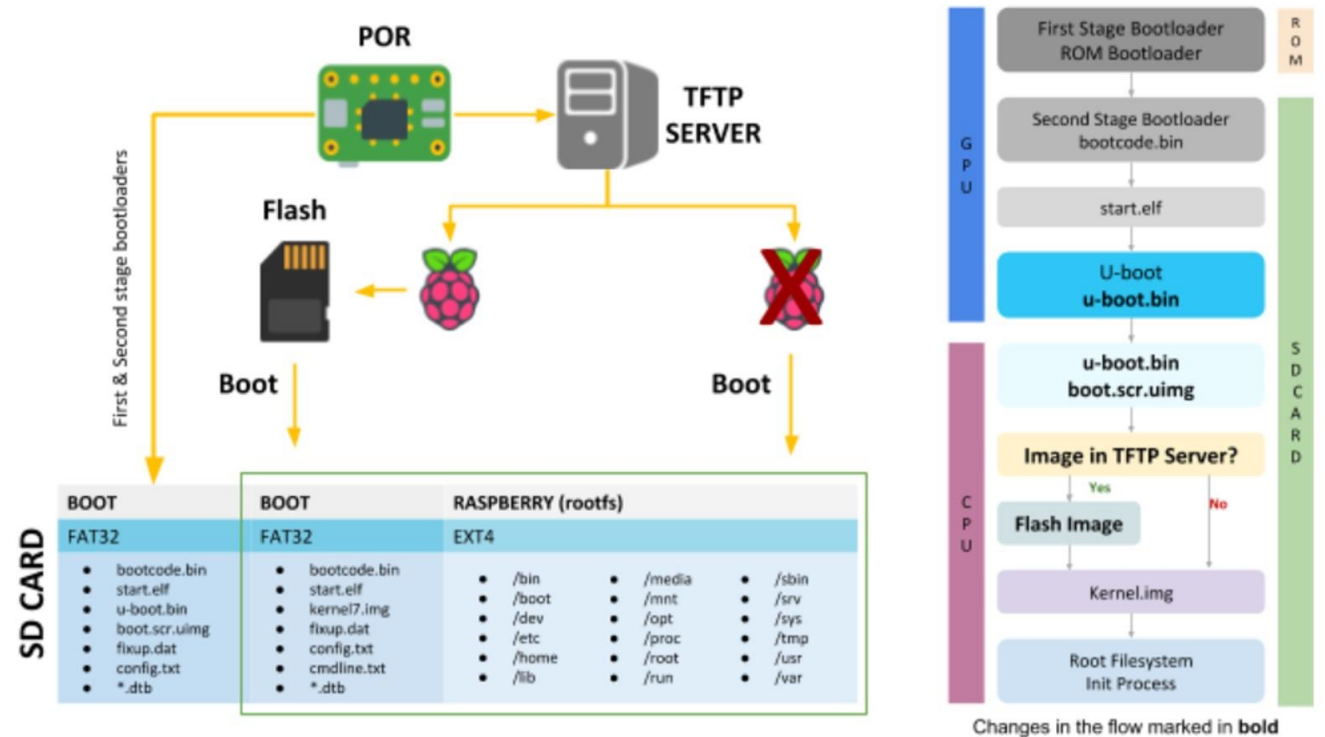


Figure 5.16: Raspberry Pi Remote Flashing

— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. Device drivers
2. BIOS
3. The boot process
- 4. Operating systems**
5. Processes and threads
6. Multiprocessing

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

- An operating system is a **multilayer suite of software**, providing an **environment** in which applications perform useful functions such as *word processing, placing telephone calls, or managing the operation of a car engine*.
- Applications running within the operating system execute algorithms implemented as processor instruction sequences and perform I/O interactions with peripheral devices as required to complete their tasks.
- The operating system provides **standardized programming interfaces** that application developers use to access **system resources** such as *processor execution threads, disk files, input* from a keyboard or other peripherals, and *output* to devices such as a computer screen or instruments on a dashboard.
- Operating systems can be broadly categorized into **real-time** and **non-real-time** systems.

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

- A **real-time operating system (RTOS)** provides features to ensure that responses to inputs occur within a defined time limit.
- Processors performing tasks such as managing the operation of a car engine or a kitchen appliance typically run an RTOS to ensure that the electrical and mechanical components they control receive responses to any change in inputs within a bounded time.
- **Non-real-time** operating systems do not attempt to ensure that responses are generated within any particular time limit. Instead, these systems attempt to perform the processing as quickly as possible, even if it sometimes takes a long time.
- Operating System Market Share Worldwide: <https://gs.statcounter.com/os-market-share>

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

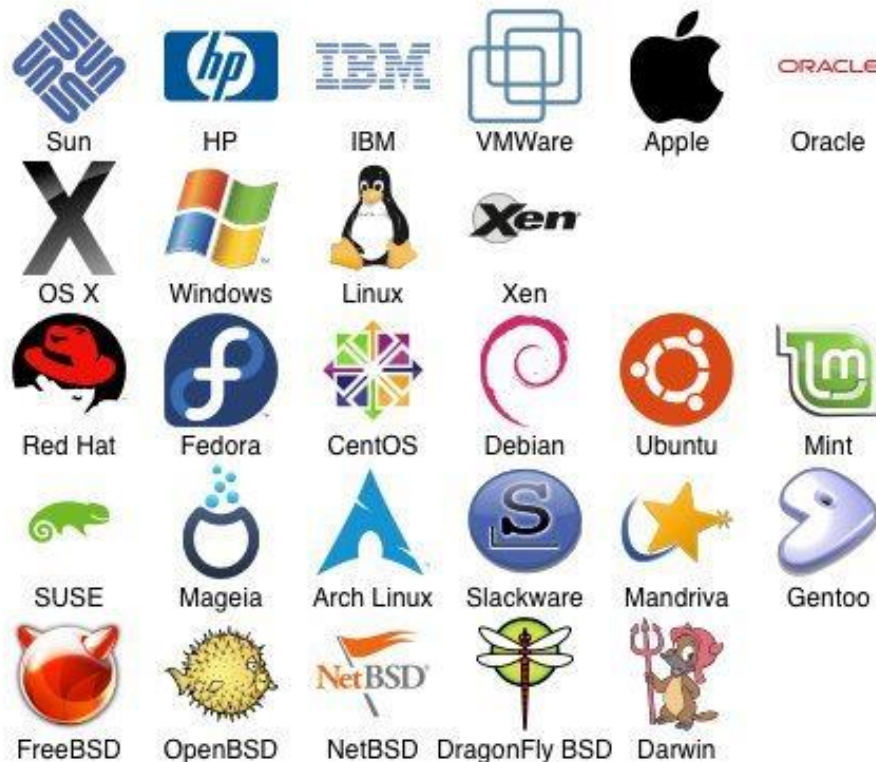


Figure 5.17: examples of Operating Systems



Figure 5.18: functions of an Operating System

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

Real-time versus non-real-time operating systems

RTOSes are not necessarily faster than non-real-time operating systems. A non-real-time operating system may be faster on average compared to an RTOS, but the non-real-time system may occasionally exceed the time limit specified for the RTOS. The goal of the RTOS is to never exceed the response time limit.

Figure 5.19: Real-time versus non-real-time operating systems

- For the most part, general-purpose operating systems such as **Windows** and **Linux** are **non-real-time** operating systems. They try to get assigned tasks—such as reading a file into a word processor or computing a spreadsheet—finished as quickly as possible, though the time to complete an operation may vary widely, depending on what other tasks the system is performing.
- Some aspects of general-purpose operating systems, particularly **audio** and **video output**, have specific **real-time** requirements.
- We've all seen poor video replay at one time or another, in which the video stutters and appears jerky. This behavior is the result of failing to meet the real-time performance demands of video display.
- **Cell phones** have similar **real-time requirements** for supporting **two-way audio during voice telephone calls**.
- For both real-time and non-real-time operating systems, in standard PCs as well as in embedded devices, operating system startup tends to follow a **similar sequence of steps**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

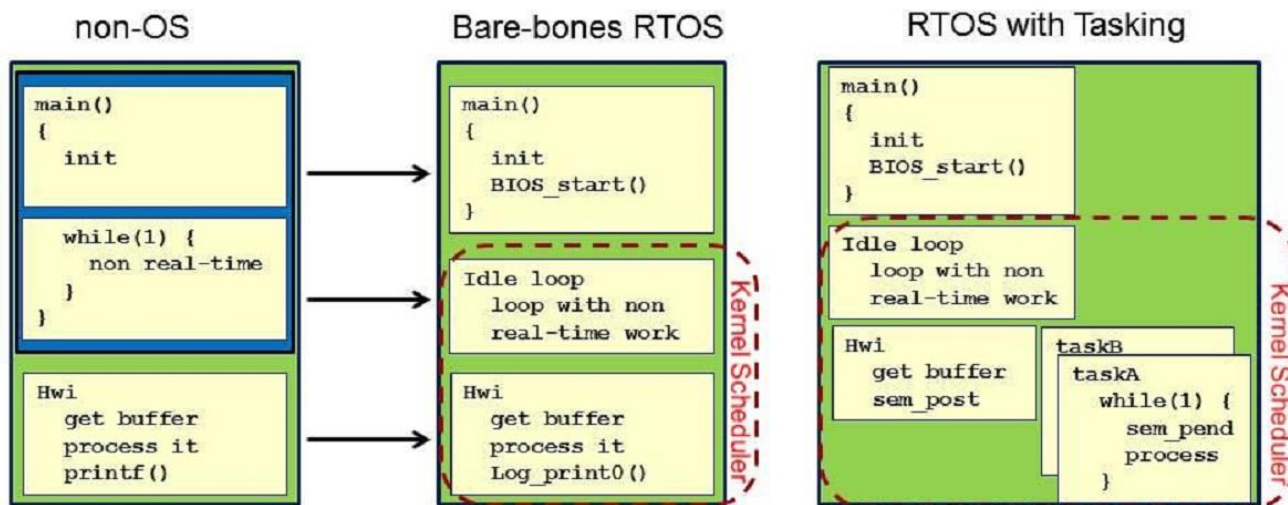


Figure 5.20: RTOS versus OS

Structure of RTOS

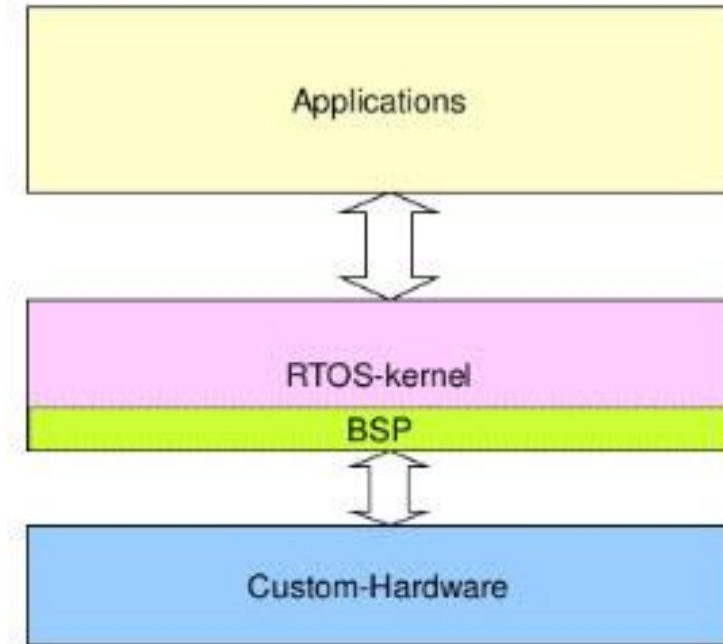


Figure 5.21: structure of RTOS

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

- The **boot loader** either *loads* the **operating system kernel** into memory or simply jumps to an address in NVM to begin executing operating system code.
- The **operating system kernel** performs the following steps, not necessarily in this order:
 - The **processor** and other **system devices** are **configured**. This includes setting up any required **registers** internal to the **processor** and any associated I/O management devices, such as a **chipset**.
 - In systems using paged virtual memory (introduced in *Chapter 7, Processor and Memory Architectures*), the **kernel** configures the **memory management unit**.
 - Base-level system processes, including the **scheduler** and the **idle process**, are started. The **scheduler** manages the sequence of execution for process threads. The **idle process** contains the code that executes when there are no other threads ready for the scheduler to run.
 - ...

— Computer Architecture

Chapter 5: Hardware-Software Interface

4. Operating systems

- The **operating system kernel** performs the following steps, not necessarily in this order:
 - ...
 - **Device drivers** are **enumerated** and associated with each peripheral in the system. Initialization code is executed for each driver, as discussed earlier in this chapter.
 - **Interrupts** are **configured** and **enabled**. Once interrupts have been enabled, the system begins to perform I/O interactions with peripheral devices.
 - **System services** are **started**. These processes support non-operating system activities (such as networking) and persistent installed capabilities (such as a web server).
 - For PC-type computers, a **user interface process** is started, which presents a login screen. This screen allows the user to initiate an interactive session with the computer. In **embedded devices**, the **real-time** application begins execution. The basic operational sequence for a simple embedded application is to read inputs from I/O devices, execute a computation algorithm to generate outputs, and write the outputs to I/O devices, repeating this procedure at fixed time intervals.

— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. Device drivers
2. BIOS
3. The boot process
4. Operating systems
- 5. Processes and threads**
6. Multiprocessing

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- Many, but not all, operating systems support the concept of **multiple threads of execution**.
- A **thread** is a sequence of program instructions that logically executes in *isolation* from other threads.
- An operating system running on a single-core processor creates the illusion of multiple simultaneously running threads with **time-slicing**.
- When performing *time-slicing*, an operating system scheduler grants each ready-to-run thread a period of time in which to execute.
- As a thread's execution interval ends, the scheduler interrupts the running thread and continues executing the next thread in its queue. In this manner, the scheduler gives each thread a bit of time to run before going back to the beginning of the list and starting over again.
- In operating systems capable of supporting multiple runnable programs simultaneously, the term **process** refers to *a running instance of a computer program*. The system allocates resources such as memory and membership in the scheduler's queue of runnable threads to each process.

Computer Architecture

Chapter 5: Hardware-Software Interface

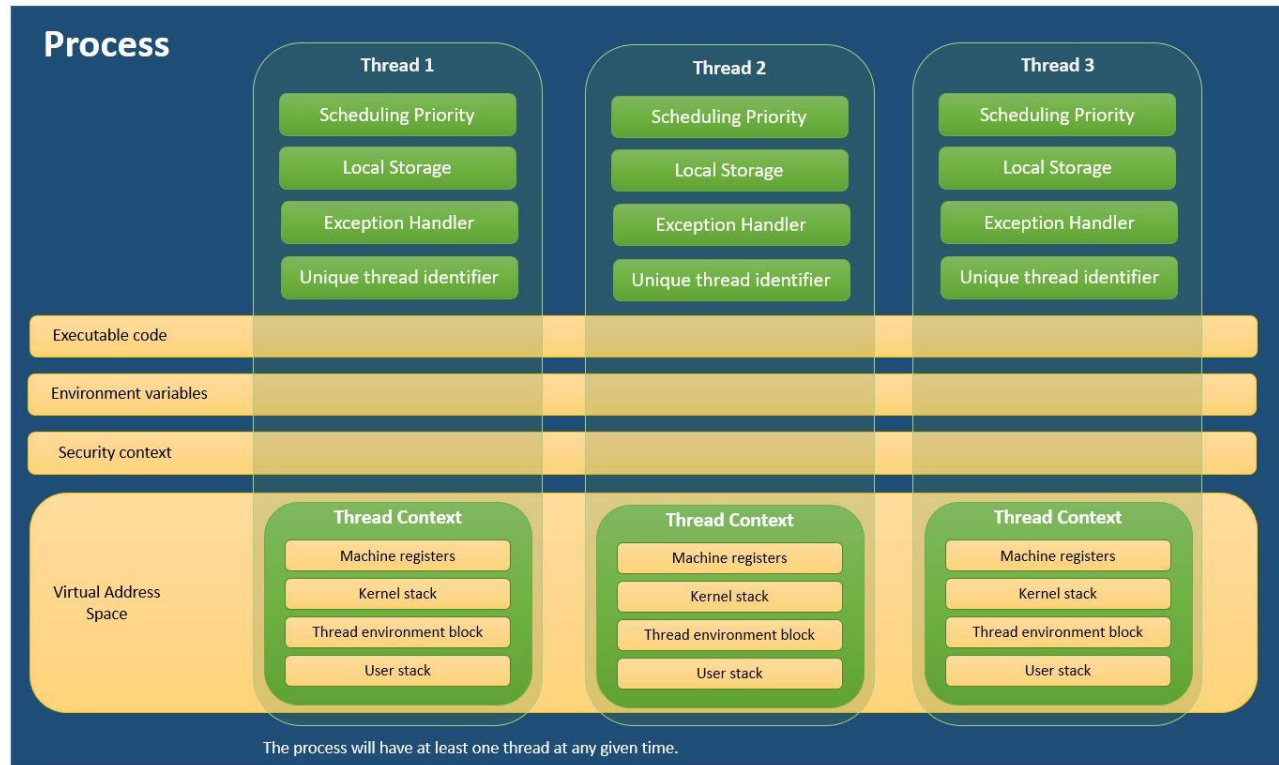


Figure 5.22: Process versus Threads

13

Threads vs. Processes

A thread has no data segment or heap	A process has code/data/heap & other segments
A thread cannot live on its own, it must live within a process	There must be at least one thread in a process
There can be more than one thread in a process, the first thread calls main & has the process's stack	Threads within a process share code/data/heap, share I/O, but each has its own stack & registers
Inexpensive creation	Expensive creation
Inexpensive context switching	Expensive context switching
If a thread dies, its stack is reclaimed	If a process dies, its resources are reclaimed & all threads die

Figure 5.23: Threads versus Processes

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- When a **process** first begins execution, it contains a **single thread**.
- The process may create more threads as it executes.
- Programmers create **multithread applications** for various reasons, including the following:
 - One thread can perform **I/O** while a separate thread executes the **main algorithm** of the program. For example, a primary thread can periodically update a user **display** with received information while a separate thread waits in a blocked state for user input from the **keyboard**.
 - Applications with substantial computational requirements can take advantage of multiprocessor and multi-core computer architectures by **splitting large computational jobs into groups of smaller tasks** capable of execution in **parallel**. By running each of these smaller tasks as a separate thread, programs enable the scheduler to assign threads *to execute on multiple cores simultaneously*.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- A **process** passes through a series of **states** during its life cycle.
- Some **process states** assigned by operating systems are as follows:
 - **Initializing**: When a process is first started, perhaps as the result of a user double-clicking an icon on the desktop, the operating system begins loading the program code into memory and assigning system resources for its use.
 - **Waiting**: After process initialization has completed, it is ready to run. At this point, its thread is assigned to the **scheduler's queue of runnable threads**. The process remains in the *Waiting* state until the scheduler permits it to start running.
 - **Running**: The thread executes the program instructions contained in its code section.
 - **Blocked**: The thread enters this state when it requests I/O from a device that causes execution to pause. For example, reading data from a file normally causes **blocking**. In this state, the thread waits in the *Blocked* state for the device driver to finish processing the request. As soon as a running thread becomes blocked, the scheduler is free to switch to another runnable thread while the first thread's I/O operation is in progress. When the operation completes, the blocked thread returns to the **Waiting state** in the scheduler queue and eventually returns to the Running state, where it processes the results of the I/O operation.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

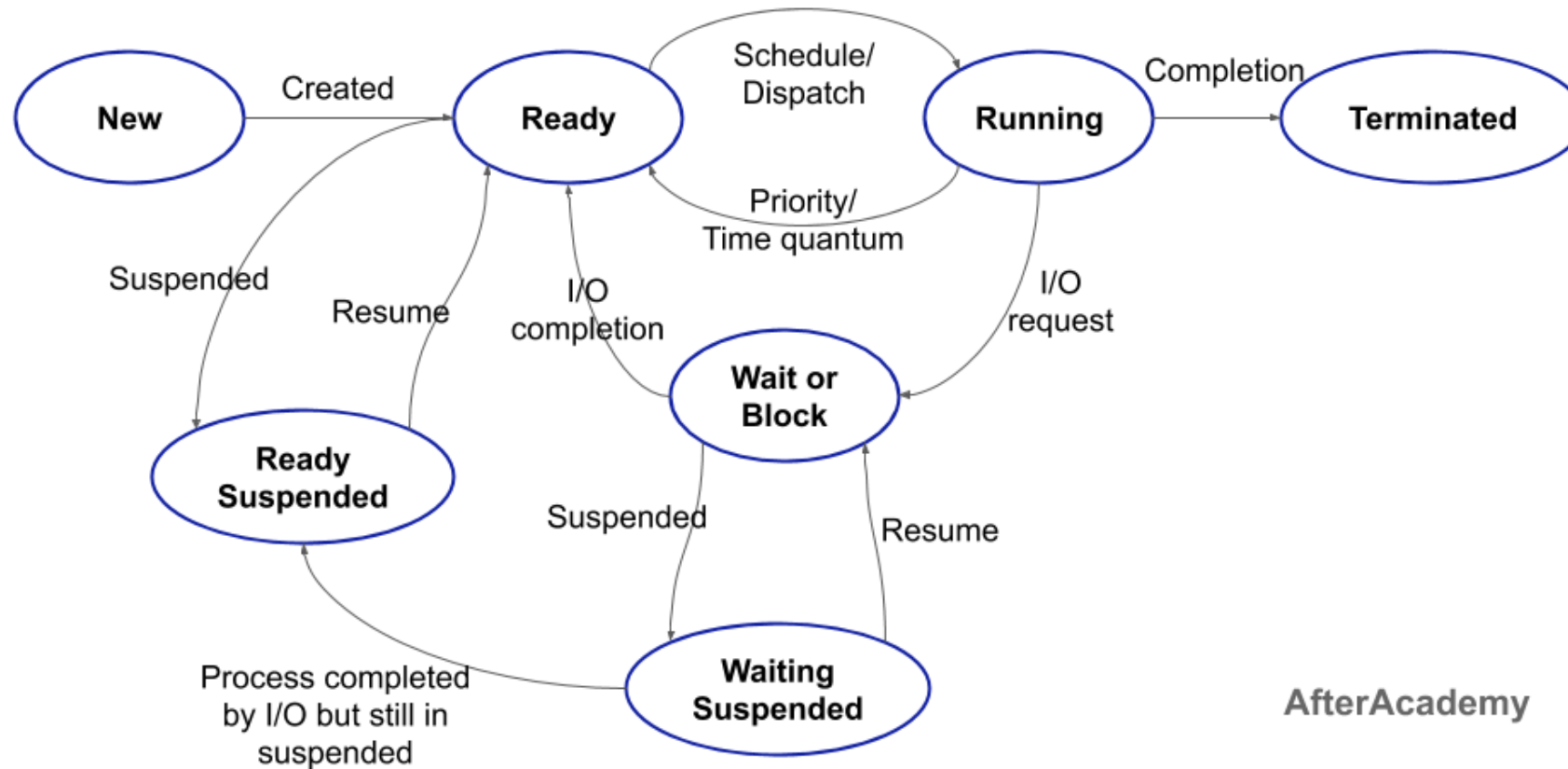


Figure 5.24: Process states

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- Ready-to-run processes rely on the **system scheduler** to receive execution time.
- The **scheduler process** is responsible for **granting execution time** to all system and user threads.
- The **scheduler** is an **interrupt-driven** routine that executes at periodic time intervals, as well as in response to actions taken by threads such as the initiation of an I/O operation.
- During **operating system initialization**, a **periodic timer** is attached to the scheduler interrupt handler, and the scheduler timer is started.
- While each process is in the *Initializing* state, the kernel adds a data structure called a **process control block (PCB)** to its list of running processes. The PCB contains information that the system requires to maintain and interact with the process over its lifetime, including memory allocations and details regarding the file containing its executable code.
- A process is usually identified by an integer that remains unique during its lifetime. In **Windows**, the **Resource Monitor** tool (**Broncontrole**) (*you can start this tool by typing Resource Monitor into the Windows search box and clicking on the result identified as Resource Monitor*) displays running processes, including the process identifier (PID) associated with each process.
- In **Linux**, the **top** command displays the processes consuming the most system resources, identifying each by its PID.

Computer Architecture

Chapter 5: Hardware-Software Interface

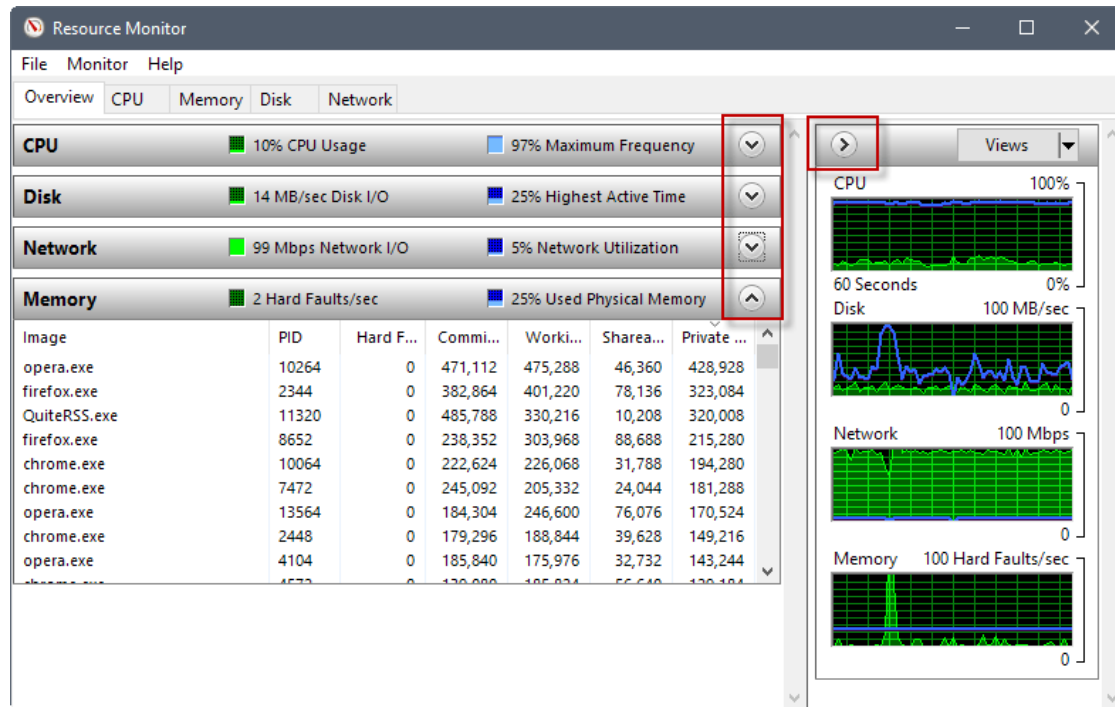


Figure 5.25: Windows Resource Monitor

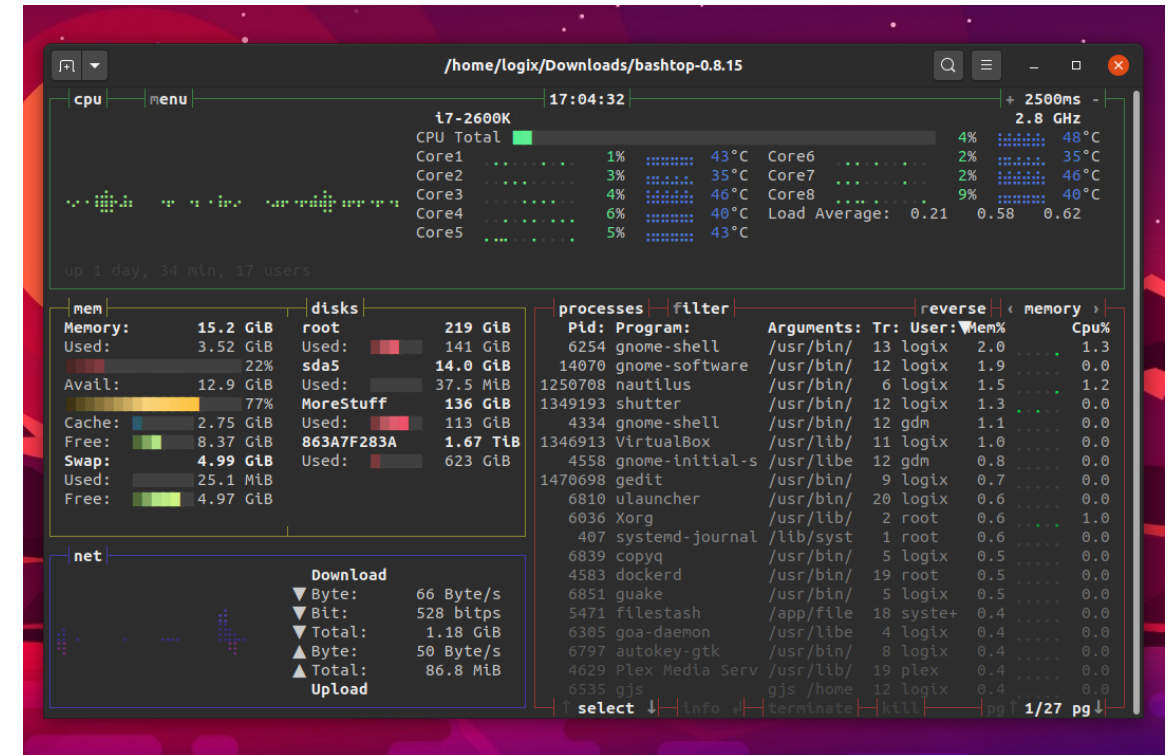


Figure 5.26: Linux Resource Monitor

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The **scheduler** maintains information associated with each thread in a **thread control block (TCB)**.
- Each process has a list of associated TCBs, with a minimum of one entry in the list.
- The TCB contains information related to the thread, including **processor context**.
- The **processor context** is the collection of information the kernel uses to resume execution of a blocked thread, consisting of these items:
 - The saved **processor registers**
 - The **stack pointer**
 - The **flags register**
 - The **instruction pointer**
- Similar to the **PID**, each thread has an integer thread identifier that remains unique during its lifetime.

Computer Architecture

Chapter 5: Hardware-Software Interface

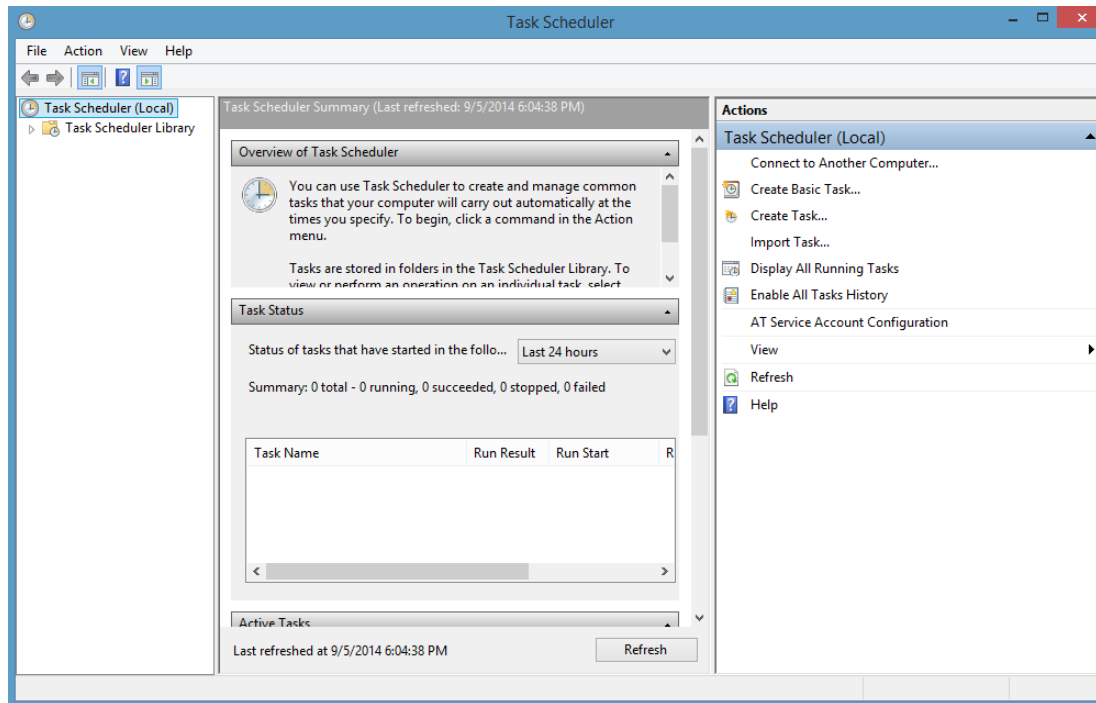


Figure 5.27: Windows Task Scheduler (Taakbeheer)

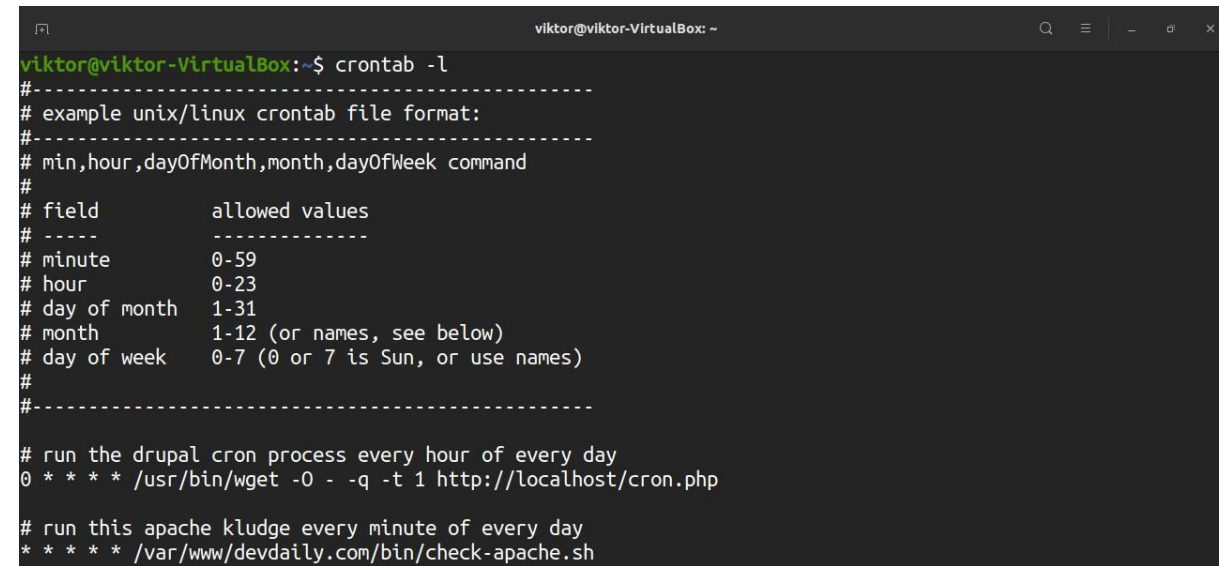


Figure 5.28: Linux Task Scheduler (crontab)

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The **scheduler** uses one or more **scheduling algorithms** to ensure that execution time is allocated equitably among system and user processes.
- Two main categories of thread scheduling algorithms have been widely used since the early days of computing—**non-preemptive** and **preemptive**, described here:
 - **Non-preemptive scheduling** grants a thread complete execution control, allowing it to run until it terminates, or voluntarily releases control to the scheduler so that other threads have a chance to run.
 - In **preemptive scheduling**, the scheduler has the authority to stop a running thread and hand execution control over to another thread without requesting approval from the first thread.

— Computer Architecture

Chapter 5: Hardware-Software Interface

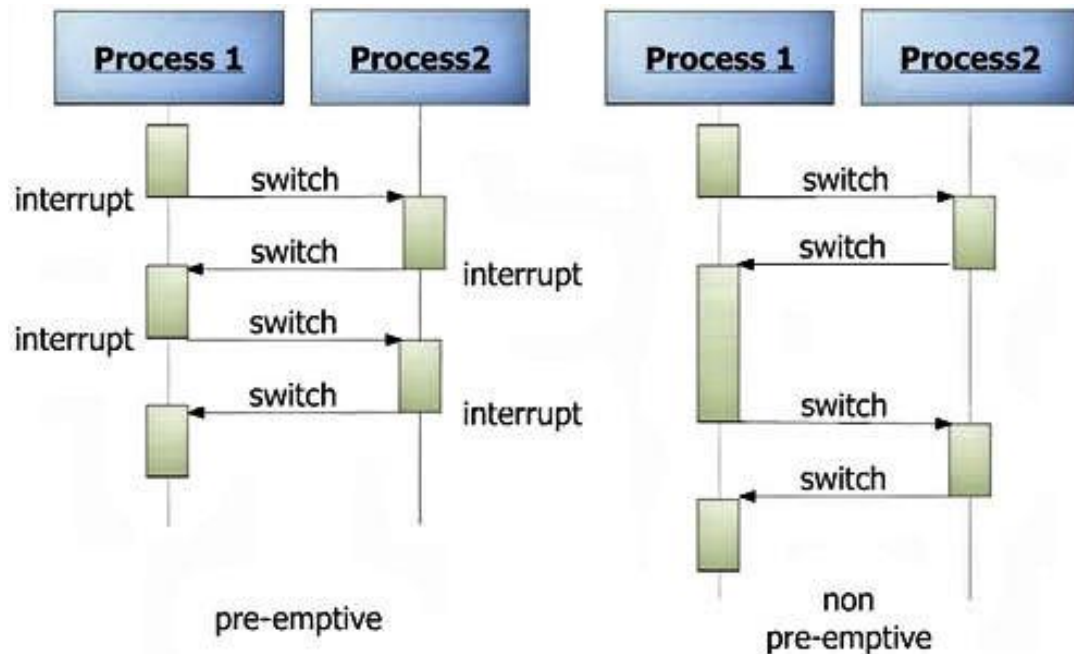
5. Processes and threads

- When a **preemptive scheduler** switches execution from one thread to another, it performs the following steps:
 1. Either a **timer interrupt** occurs that causes the scheduler to begin execution, or a running thread performs an action that causes **blocking**, such as initiating an I/O operation.
 2. The scheduler consults its **list of runnable threads** and determines which thread to place in the Running state.
 3. The scheduler *copies* the departing thread's processor registers into the context fields of the thread's **TCB**.
 4. The scheduler *loads* the context of the incoming thread into the processor registers.
 5. The scheduler *resumes execution* of the incoming thread by jumping to the instruction pointed to by the thread's program counter.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads



Preemptive vs. Non-preemptive scheduling

■ Non-preemptive scheduling:

- The running process keeps the CPU until it **voluntarily** gives up the CPU
 - process exits
 - switches to blocked state
 - Transition 3 is only voluntary

■ Preemptive scheduling:

- The running process can be interrupted and must release the CPU (can be **forced** to give up CPU)

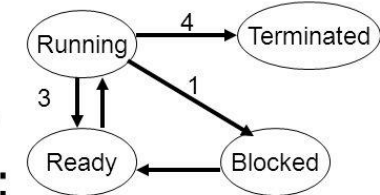


Figure 5.29: pre-emptive versus non pre-emptive scheduling

Figure 5.30: non-preemptive versus preemptive scheduling

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- **Thread scheduling** occurs at a high frequency, which implies the code involved in scheduler activity must be as efficient as possible.
- In particular, storing and retrieving processor context takes some time, so operating system designers make every effort to optimize the performance of the scheduler's context switching code.
- Because there may be numerous processes competing for execution time at a given moment, the scheduler must ensure that critical system processes are able to execute at required intervals. At the same time, from the user's point of view, applications must remain responsive to user inputs while providing an acceptable level of performance during lengthy computations.
- Various **algorithms** have been developed over the years to efficiently manage these competing demands. A key feature of most thread scheduling algorithms is the use of **process priorities**.
- The next section introduces several **priority-based thread scheduling algorithms**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads: Scheduling algorithms and process priority

- Operating systems supporting multiple processes generally provide a **prioritization mechanism** to ensure that the most important system functions receive adequate processing time, even when the system is under heavy load, while continuing to provide adequate time for the execution of lower-priority user processes.
- Several **algorithms** have been developed to meet various performance goals for different types of operating systems.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads: Scheduling algorithms and process priority

- Some algorithms that have been popular over the years, beginning with the simplest, are as follows:
 - **First come, first served (FCFS)**: This **non-preemptive** approach was common in *legacy batch processing operating systems*. In an FCFS scheduling algorithm, each process is granted execution control and retains control until execution is completed. There is **no prioritization of processes**, and the time to complete any process is dependent on the execution time of processes preceding it in the input queue.
 - **Cooperative multithreading**: *Early versions of Windows and macOS* used a **non-preemptive multithreading architecture** that relied on each thread to voluntarily relinquish control to the operating system at frequent intervals. This required significant effort by application developers to ensure a single application did not starve other applications of opportunities to execute by failing to release control at appropriate intervals. Each time the operating system received control, it selected the next thread to execute from a **prioritized list of runnable threads**.
 - **Round-robin scheduling**: A **preemptive round-robin scheduler** maintains a list of runnable threads and grants each of them an execution interval in turn, starting over at the beginning of the list upon reaching the end. This approach effectively sets **all process priorities equally**, giving each a chance to execute for defined periods of time, at time intervals dependent on the number of processes in the scheduler's list.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads: Scheduling algorithms and process priority

- Some algorithms that have been popular over the years, beginning with the simplest, are as follows:
 - **Fixed-priority preemptive scheduling:** In this algorithm, each thread is assigned a **fixed priority** value indicating the importance of its receiving execution control when it is in the *Waiting* state. When a thread enters the **Waiting** state, if it has a **higher** priority than the currently running thread, the scheduler immediately stops the running thread and turns control over to the incoming thread. The scheduler maintains the *list* of Waiting processes in priority order, with *the highest priority threads at the head of the line*. This algorithm can result in the *failure of lower-priority threads* to get any execution time at all if higher-priority threads monopolize the available execution time.
 - **Rate-monotonic scheduling (RMS)** is a **fixed-priority preemptive** scheduling algorithm commonly used in **real-time systems** with hard deadlines (a hard deadline is one that cannot be missed). Under RMS, **threads that execute more frequently are assigned higher priorities**. As long as a few criteria are satisfied (the thread execution interval equals the deadline; there can be no delay-inducing interactions between threads; and context switch time is negligible), if the maximum possible execution time of each thread is below a mathematically derived limit, **deadlines are guaranteed to be met**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads: Scheduling algorithms and process priority

- Some algorithms that have been popular over the years, beginning with the simplest, are as follows:
 - **Fair scheduling**: Fair scheduling attempts to maximize the utilization of processor time while ensuring that **each user is granted an equal amount of execution time**. Rather than assigning numeric priorities to threads, the effective **priority** of each thread is *determined by the amount of execution time it has consumed*. As a thread uses more and more processor time, its priority declines, enabling other threads more opportunities to run. This approach has the benefit that, for interactive users who do not consume much execution time, the responsiveness of the system is improved. The **Linux** kernel uses a **fair scheduling algorithm** as its default scheduler.
 - **Multilevel feedback queue**: This algorithm uses **multiple queues**, each with a *different priority level*. New threads are added at the tail of the highest-priority queue. At each scheduling interval, the scheduler grants execution to the thread at the head of the high-priority queue and removes the thread from the queue, which moves the remaining threads closer to execution. Eventually, the newly created thread receives an opportunity to execute. If the thread consumes all the execution time granted to it, it is preempted at the completion of its interval and added to the tail of the next lower-priority queue. The **Windows Scheduler** is a **multilevel feedback queue**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The **system idle process** contains the thread that executes when there is *no user- or system-assigned thread* in the *Waiting* state. An idle process may be as simple as a single instruction that forms an infinite loop, jumping to itself.
- Some operating systems place the system in a **power-saving mode** during idle periods, rather than executing an idle loop.
- The percentage of processor time consumed by running processes is computed by determining the fraction of time the system was executing a non-idle thread over a measurement period.

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The following screenshot is a **Windows Resource Monitor** view of running processes consuming the highest average share of processor time (**Figure 5.31**).
- In this figure, the **PID** column displays the numeric **process identifier**, and the **Threads** column shows the **number of threads in the process**.
- The **process state** is **Running** for all of the processes visible in this display.

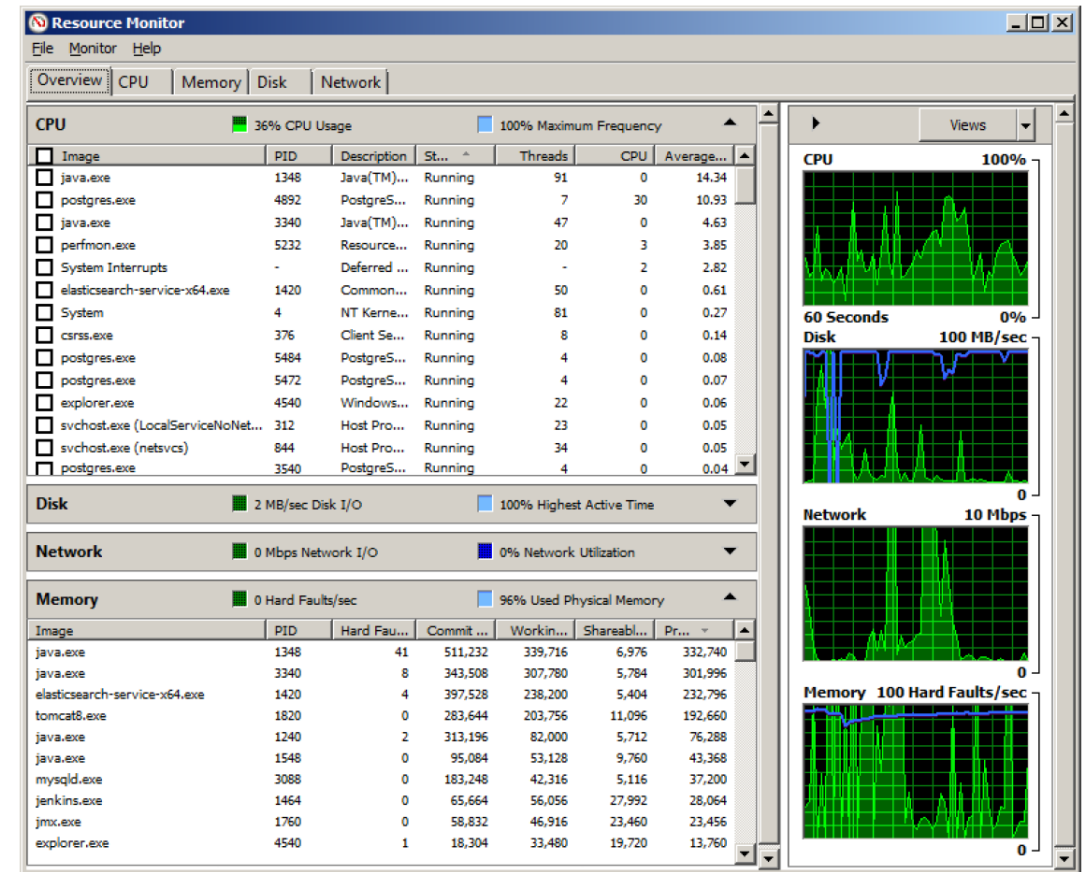


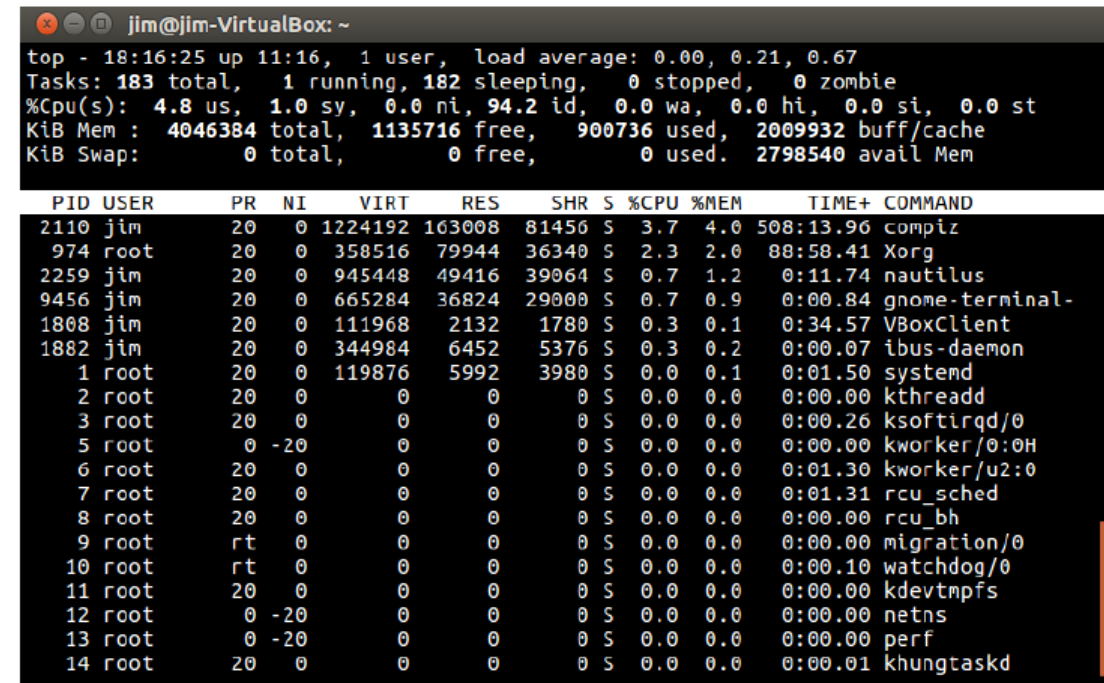
Figure 5.31: Windows Resource Monitor

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The following screenshot shows the result of running the top command on a **Linux** system (Figure 5.32).
- The upper part of the display contains summary information, including the number of processes (referred to as **Tasks** here) in each state.
- Each row in the lower part of the display presents information about one running process.
- As in **Windows**, the **PID** column indicates the PID.



```
top - 18:16:25 up 11:16, 1 user, load average: 0.00, 0.21, 0.67
Tasks: 183 total, 1 running, 182 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.8 us, 1.0 sy, 0.0 ni, 94.2 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 4046384 total, 1135716 free, 900736 used, 2009932 buff/cache
KiB Swap: 0 total, 0 free, 0 used. 2798540 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2110	jim	20	0	1224192	163008	81456	S	3.7	4.0	508:13.96	compiz
974	root	20	0	358516	79944	36340	S	2.3	2.0	88:58.41	Xorg
2259	jim	20	0	945448	49416	39064	S	0.7	1.2	0:11.74	nautilus
9456	jim	20	0	665284	36824	29000	S	0.7	0.9	0:00.84	gnome-terminal-
1808	jim	20	0	111968	2132	1780	S	0.3	0.1	0:34.57	VBoxClient
1882	jim	20	0	344984	6452	5376	S	0.3	0.2	0:00.07	ibus-daemon
1	root	20	0	119876	5992	3980	S	0.0	0.1	0:01.50	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.26	ksoftirqd/0
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/0:0H
6	root	20	0	0	0	0	S	0.0	0.0	0:01.30	kworker/u2:0
7	root	20	0	0	0	0	S	0.0	0.0	0:01.31	rcu_sched
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_bh
9	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/0
10	root	rt	0	0	0	0	S	0.0	0.0	0:00.10	watchdog/0
11	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
12	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	netns
13	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	perf
14	root	20	0	0	0	0	S	0.0	0.0	0:00.01	khungtaskd

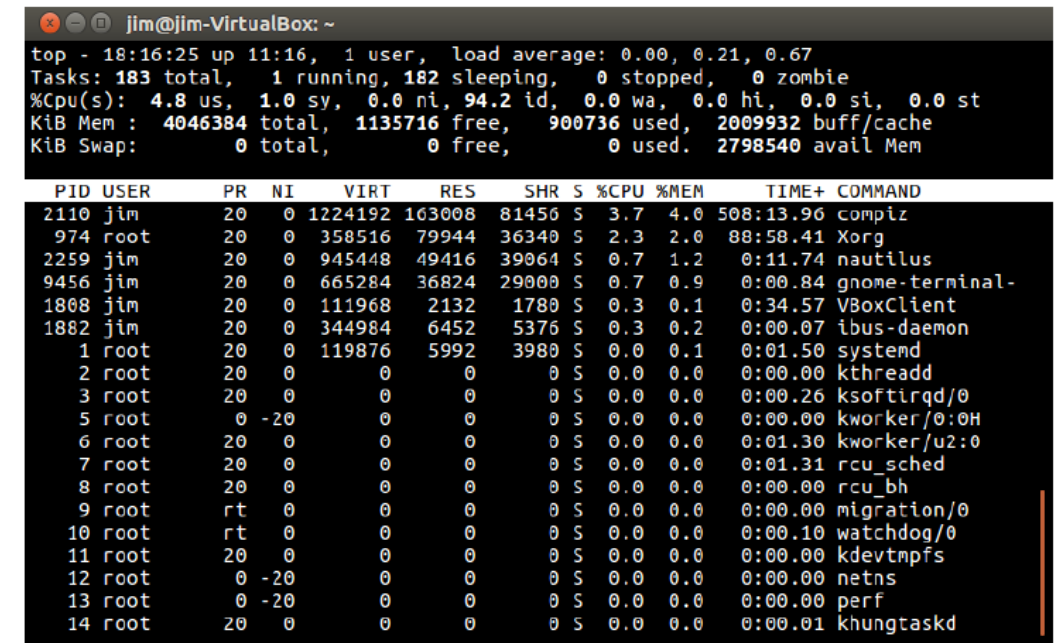
Figure 5.32: Linux scheduler

— Computer Architecture

Chapter 5: Hardware-Software Interface

5. Processes and threads

- The **state** of each process is shown in the **S column**, with these possible values:
 - R: Runnable**, meaning either running or in the queue of ready-to-run threads.
 - S: Sleeping**: Paused while blocked; waiting for an event to complete.
 - T: Stopped** in response to a job control command (pressing CTRL+Z will do this).
 - Z: Zombie**, which occurs when a child process of another process terminates, but the child process information continues to be maintained by the system until the parent process ends.
 - PR**: The PR column displays the **scheduling priority** of the process. Smaller numbers represent higher priorities.



PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2110	jim	20	0	1224192	163008	81456	S	3.7	4.0	508:13.96	compiz
974	root	20	0	358516	79944	36340	S	2.3	2.0	88:58.41	Xorg
2259	jim	20	0	945448	49416	39064	S	0.7	1.2	0:11.74	nautilus
9456	jim	20	0	665284	36824	29000	S	0.7	0.9	0:00.84	gnome-terminal-
1808	jim	20	0	111968	2132	1780	S	0.3	0.1	0:34.57	VBoxClient
1882	jim	20	0	344984	6452	5376	S	0.3	0.2	0:00.07	ibus-daemon
1	root	20	0	119876	5992	3980	S	0.0	0.1	0:01.50	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.26	ksoftirqd/0
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/0:0H
6	root	20	0	0	0	0	S	0.0	0.0	0:01.30	kworker/u2:0
7	root	20	0	0	0	0	S	0.0	0.0	0:01.31	rcu_sched
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_bh
9	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/0
10	root	rt	0	0	0	0	S	0.0	0.0	0:00.10	watchdog/0
11	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
12	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	netns
13	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	perf
14	root	20	0	0	0	0	S	0.0	0.0	0:00.01	khungtaskd

Figure 5.32: Linux scheduler

— Computer Architecture

Chapter 5: Hardware-Software Interface

Contents

1. Device drivers
2. BIOS
3. The boot process
4. Operating systems
5. Processes and threads
- 6. Multiprocessing**

— Computer Architecture

Chapter 5: Hardware-Software Interface

6. Multiprocessing

- A **multiprocessing computer** contains two or more processors that *simultaneously* execute sequences of instructions.
- The processors in such a system typically *share access to system resources*, such as **main memory** and **peripheral devices**.
- The processors in a multiprocessing system may be of the same architecture, or individual processors may be of differing architectures to support unique system requirements.
- Systems in which all processors are treated as **equal** are referred to as **symmetric multiprocessing systems (SMP)**.
- Devices that contain multiple processors within a single integrated circuit are called **multi-core processors**.

— Computer Architecture

Chapter 5: Hardware-Software Interface

6. Multiprocessing

- At the level of the operating system scheduler, a **symmetric multiprocessing environment** simply provides more processors for use in **thread scheduling**.
- In such systems, the scheduler treats additional processors as *resources* when assigning threads for execution.
- In a well-designed **symmetric** multiprocessing system, throughput can approach the ideal of **scaling linearly** with the number of available processor cores, as long as **contention** for shared resources is minimal.
- If multiple threads on separate cores attempt to perform heavy simultaneous access to main memory, for example, there will be inevitable performance degradation as the system arbitrates access to the resource and shares it among competing threads.
- A **multichannel interface** to dynamic random-access memory (DRAM) can improve system performance in this scenario.

— Computer Architecture

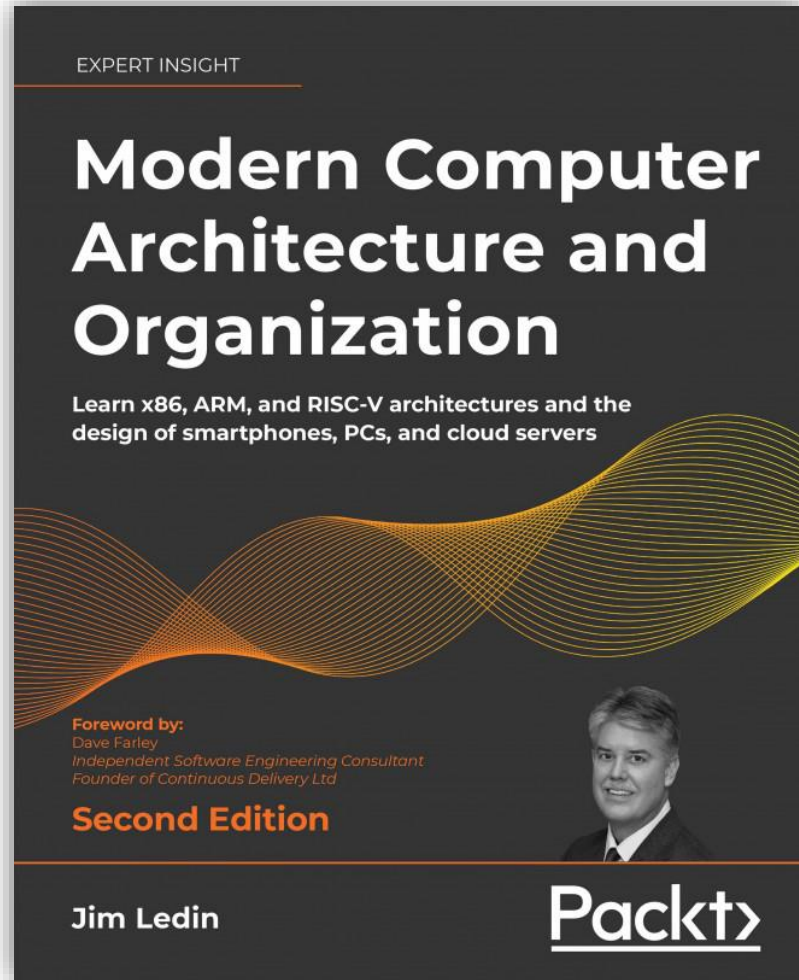
Chapter 5: Hardware-Software Interface

6. Multiprocessing

- A **symmetric multiprocessing system** is an example of a **multiple instruction, multiple data (MIMD)** architecture. **MIMD** is a **parallel** processing configuration in which each processor core executes an independent sequence of instructions on its own set of data.
- A **single instruction, multiple data (SIMD)** parallel processing configuration, on the other hand, performs the same instruction operation on multiple data elements simultaneously. Modern processors implement SIMD instructions to perform parallel processing on large datasets such as **graphical images** and **audio data sequences**.
- In current-generation PCs, the use of **multi-core processors** enables **MIMD** execution parallelism, while specialized instructions within the processors provide a degree of SIMD execution parallelism. SIMD processing will be discussed further in *Chapter 8, Performance-Enhancing Techniques*.
- Processor clock speeds have grown from the 4.77 MHz of the original PC to over 4 GHz in modern processors, nearly a thousand-fold increase.
- Future speed increases are likely to be more **limited** as **fundamental physical limits** present looming obstacles.
- To compensate for limited future performance gains from increases in clock speed, the processor industry has turned to emphasizing various forms of **execution parallelism** in personal computer systems and smart devices.
- Future trends are likely to continue the growth in **parallelism** as systems integrate dozens, then hundreds, and eventually thousands of processor cores executing in parallel in PCs, smartphones, and other digital devices.

Computer Architecture

End of Chapter 5 – Hardware-Software Interface



VIVES University of Applied Sciences
Bachelor in electronics-ICT

